



M363 Unit 8

UNDERGRADUATE COMPUTING

Software engineering with objects



The case study: Part 2

Unit **8**

This publication forms part of an Open University course M363 *Software engineering with objects*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc; Motif is a registered trademark of the Open Group; Rational Unified Process is a registered trademark of International Business Machines Corporation; Unified Modeling Language and UML are trademarks of Object Management Group, Inc. Design by Contract is a trademark of Interactive Software Engineering. Other product and company names may appear in the M363 course material. Rather than use a trademark symbol with every occurrence of a trademarked name, we use the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed in the United Kingdom by Thanet Press Ltd, Margate

ISBN 978 0 7492 1614 6

An abstract graphic on the left side of the page features several interlocking, translucent rings or bands that appear to be floating above a body of water. The water's surface is covered in ripples, and the entire scene is bathed in a blue light, creating a serene and technological atmosphere. The rings have a metallic or glass-like texture and are arranged in a way that suggests movement and interconnectedness.

CONTENTS

1	Introduction	5
2	Domain modelling – the concept diagram	8
2.1	Identifying concepts, associations, multiplicities and attributes	8
2.2	Summary of section	16
3	Analysis	17
3.1	Constraints	17
3.2	System operations	20
3.3	Specifying operations	21
3.4	Summary of section	31
4	Design – assigning responsibilities	32
4.1	Value identities	32
4.2	Verifying and fulfilling pre- and postconditions	33
4.3	Revising the class diagram	43
4.4	Summary of section	44
5	Summary	45
	Reference	47
	Index	48

M363 COURSE TEAM

Robin Laney, Course Chair, Author and Academic Editor

Leonor Barroca, Author

Ralph Greenwell, Course Manager

Charles Haley, Author

Nigel Kermode, Critical Reader

Martin Shepperd, External Assessor, Brunel University

Richard Walker, Critical Reader

Andy Allum, Course Website Developer

Kim Dulson, Software Procurement

Phillip Howe, Media Assistant

Callum Lester, Software Developer

Tara Marshall, Print Procurement

Sandy Nicholson, Freelance Copy Editor

Andy Seddon, Media Project Manager

Lucinda Simpson, Editor

Sue Stavert, Technical Tester

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1

Introduction

In this unit, we will produce a **structural model**, complete the analysis of the case study, and produce the design for the first elaboration iteration of the Unified Process (UP). In *Unit 4*, from our use case diagram, we identified the *make reservation*, *cancel reservation*, *check in guest* and *check out guest* use cases as a suitable set of requirements for the first iteration. These will provide a usable slice of functionality. We also decided to focus during the first iteration on the main success scenarios.

In this unit, we will show that the analysis and design is being driven from the requirements, and establish some basic traceability between the requirements and design. We will begin by producing a structural domain model – represented as a **concept diagram**. The concept diagram is just a class diagram in which the classes represent concepts from the problem domain rather than software. This will allow us to represent sets of objects that are of relevance to the problem we are trying to solve. Although logically it is sensible to think of domain analysis preceding requirements elicitation, the existence of the requirements does not necessarily mean that we fully understand the domain. In fact this unit will show that even for an area like hotel reservations and checking in, a lot of subtle issues need to be carefully considered. Even so, where developers are working in a well known domain, they might choose to skip domain analysis and start with what needs to be represented in software.

Units 5–7 have used examples from a hotel system. A major difference in this case study is that we will conduct our analysis and create our diagrams systematically, with the whole process driven by the requirements given in *Unit 4*. So, while the examples in *Units 5–7* illustrate particular points, in the case study units we are showing the overall development approach.

In *Unit 4*, we went from an initial problem statement, through a range of activities, to a set of detailed requirements. We currently have a set of use cases for the proposed system, along with a set of software requirements.

We will start by looking at the structure of our domain, and identify which domain objects should be represented as classes, which attributes should be defined in the classes, and which relationships exist between the classes. Note that although we are considering classes, these represent concepts; we are not considering code as yet. We are still dealing with models, abstracting away from decisions related, for example, to the implementation language.

As we carry out these steps, we will produce an accompanying glossary, which defines each analysis class, its associations and attributes. In other words, the glossary documents the elements of each model.

Having completed our structural domain model, we will move on to produce the **analysis structural model**. As we already have a concept diagram, the analysis model will build on it and the requirements. Although classes in the analysis structural model will resemble those in the concept model, they represent what is needed in the software, in contrast to concept classes, which represent the real-world domain.

We adopt from Jacobson et al. (1999) the following description of the characteristics of an **analysis class**, to indicate which characteristics we should be identifying in the analysis model.

- ▶ An analysis class focuses on handling functional requirements, and postpones the handling of non-functional requirements until design and implementation.

- ▶ An analysis class seldom defines or provides any interface in terms of operations and their signatures.
- ▶ The behaviour of an analysis class may be outlined by responsibilities, although the detailed assignment of responsibilities is left until design.
- ▶ An analysis class defines attributes, although these attributes are at a fairly high level. The types of these attributes are often conceptual and recognisable from the problem domain. These attributes may become classes themselves in design and implementation.
- ▶ An analysis class is involved in relationships, although these relationships are still conceptual. Navigability is not important at this stage, although it will become essential when we come to design.

In our case, then, the concept diagram we produced can be used as the basis for the structural model for analysis. This just leaves us to consider constraints and then identify the **system operations** required from the use cases in order to complete our analysis. In identifying the system operations, there is an emphasis on their pre- and postconditions.

Moving into design, we can start deciding on the assignment of responsibilities to classes, and recording these decisions on the class diagram. This will consist of carrying out the following tasks:

- ▶ using interaction diagrams in verifying preconditions and fulfilling postconditions of the system operations – leading to identification of finer-grained operations;
- ▶ assigning the operations to classes using GRASP patterns:
 - ▶ Expert – responsibility assigned to a class with information to fulfil that responsibility;
 - ▶ Creator – responsibility given to classes that maintain, record and contain instances of the class of the newly created object;
- ▶ revisiting the class diagram to record all the decisions taken.

Figure 1 shows an overview of which parts of the **development process** we cover in this unit.

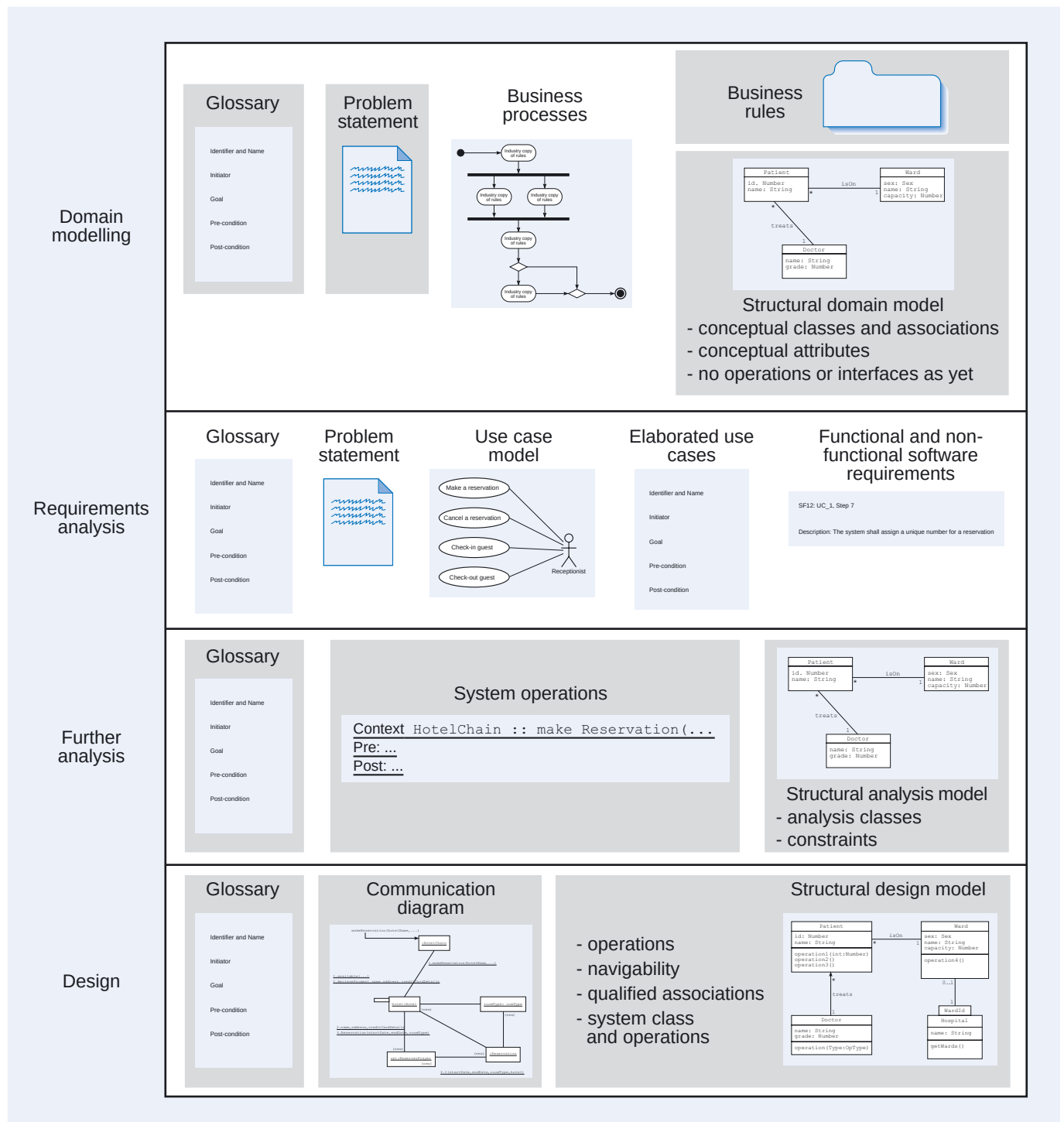


Figure 1 Parts of the development process covered in this unit

2

Domain modelling – the concept diagram

Remember that at this stage we are still describing the world in which our problem exists, rather than describing a piece of software. We begin by using the four categories of objects from *Unit 5* – tangible objects, roles, events and organisational units – to identify relevant candidate classes.

2.1

Identifying concepts, associations, multiplicities and attributes

We start by looking at the initial problem statement from *Unit 4*, Section 3, identifying potential concepts to be represented as classes, using the technique from *Unit 5*.

- ▶ Tangible objects: computer system; newspapers; debit/credit card; room.
- ▶ Roles: guest; customer; manager; receptionist.
- ▶ Events: reservation; check in; check out; cancellation; no show; hotel full; add room type/room/hotel; delete room type/room/hotel; decorating; building work; reservation request; block booking; overflow.
- ▶ Organisational units: chain of hotels; hotel; nearby hotel.

As usual when identifying concepts to be represented as classes, there is a degree of arbitrariness in the answer. For example, we could have included hotel as a tangible object instead of, or in addition to, including it as an organisational unit. However, we did not feel that this would be useful later on.

In *Unit 4*, we identified the use cases to be dealt with in the first iteration of development. In fact, from simple observation, we can see that they focus on the core functionality of the business of a hotel: checking guests in and out, and handling reservations.

We will therefore start to build our concept diagram by thinking about guests, rooms and the events that determine the relationships between them. A class diagram represents a generalisation of all the possible configurations of objects in the domain.

A concept diagram plays such a central role that it is sensible to construct it systematically, justifying each step, in order to prevent any errors creeping in. For this reason, we will begin with the relationships between guests and rooms, and because both of these classes have direct relationships with a hotel, we will also consider the hotel class.

A first attempt to produce our concept diagram can therefore borrow from Figure 4 in *Unit 5*, based on the same reasoning used there.

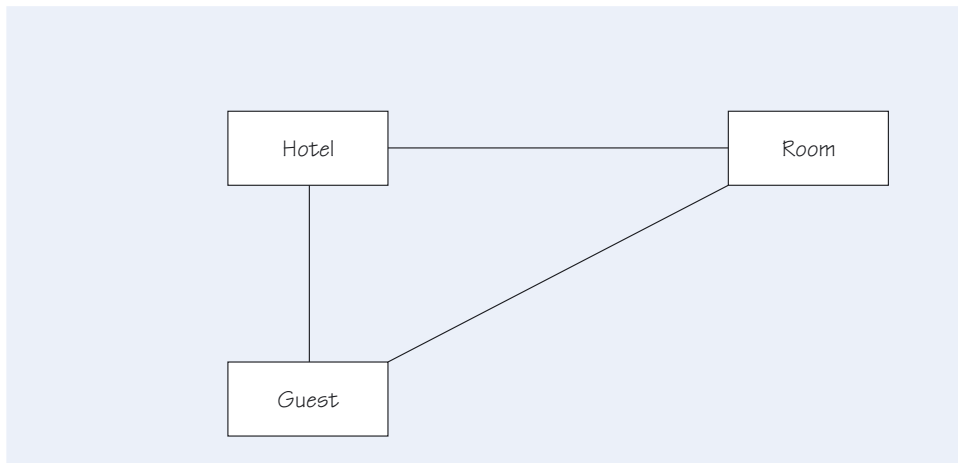


Figure 2 Some associations between *Guest*, *Hotel* and *Room*

The next step is to decide whether any other classes should be represented in the diagram, and establish how they relate to existing classes. We will do this by systematically working through the list of candidate classes given above.

We will use classes to represent the main concepts in our domain. In particular, those candidate classes that have important relationships with other classes will also become classes. However, where a candidate class is relatively simple and associated with only one other class, it can often be recorded simply as an attribute of the other class, without becoming a class in its own right.

Tables 1 to 4 show our decisions about the candidate classes listed earlier.

Table 1 Tangible objects as candidate classes

Candidate	Represent as class?
Computer system	Not part of the domain.
Newspapers	No. May record as an attribute, so that we know whether the guest should have one.
Debit/credit card	No. Will be an attribute of <i>Guest</i> .
Room	Already included.

Table 2 Roles as candidate classes

Candidate	Represent as class?
Guest	Already included.
Customer	No. Subsume under <i>Guest</i> .
Manager	No. No information needs to be represented about managers for this iteration. Although we might need to represent managers in a later iteration, we will defer this for now to concentrate on the core classes associated with this iteration.
Receptionist	No. No information needs to be represented about receptionists for this iteration.

Table 3 Events as candidate classes

Candidate	Represent as class?
Reservation	Yes, as a reservation exists over a period of time.
Check in	No. Check in alters the relationship between guests and rooms, but there is no other information that needs to be recorded.
Check out	No. Similar argument to that for check in.
Cancellation	No. Causes a reservation to be deleted, and perhaps a credit card to be debited, but again no other information needs to be recorded.
No shows	No. Similar argument to that for cancellation.
Hotel full	No. Can be represented as an attribute.
Add room type/room/hotel	No. Outside scope of existing iteration.
Delete room type/room/hotel	No. Outside scope of existing iteration.
Decorating	No. Outside scope of existing iteration.
Building work	No. Outside scope of existing iteration.
Reservation request	No. Synonym for reservation.
Block booking	No. Outside scope of existing iteration.
Overflow	No. No long-term information needs to be recorded beyond that for a normal check in.

Table 4 Organisational units as candidate classes

Candidate	Represent as class?
Hotel chain	Yes.
Hotel	Already included.
Nearby hotel	No. Hotels are already included. This information can sensibly be included as an association.

As a consequence of the decisions made in Table 3, we add *Reservation* to our list of classes for this iteration. From Table 4, we add *HotelChain*. Our classes are now:

Guest, Room, Hotel, HotelChain, Reservation.

We add entries for these classes to our (currently empty) glossary, as shown in Table 5.

Table 5 Initial glossary for the hotel system

Term	Category	Definition
<i>Guest</i>	Concept	A person who reserves a <i>Room</i> , stays in a <i>Room</i> , and is responsible for the bill. Note that the <i>Room</i> may be occupied also, for example, by the <i>Guest</i> 's family.
<i>Room</i>	Concept	A physical room in a <i>Hotel</i> .
<i>Hotel</i>	Concept	One of the hotels in the <i>HotelChain</i> .
<i>HotelChain</i>	Concept	The company which manages the <i>Hotels</i> overall.
<i>Reservation</i>	Concept	A record of the fact that a particular <i>Room</i> has been booked for a given period by a given <i>Guest</i> .

Before we attempt all the work of identifying associations and drawing the initial class diagram, it is sensible to check whether the list of classes we have identified is complete. Because we are aiming to drive the analysis and design of our system from its requirements, we do this by checking that the class diagram is able to support the use cases we identified in *Unit 4*.

SAQ 1

Work through the steps of the four use cases for the first iteration of the system as identified in *Unit 4*, considering whether there are any classes needed to support each step that we have not yet decided to include.

ANSWER.....

In UC1 *make reservation*, Step 2 refers to a type of room.

In UC3 *check in guest*, Step 6 refers to a bill.

We choose to add *RoomType* as a concept rather than as an attribute, since, as we will see later, it participates in a number of associations. Sometimes decisions like this can simply be based on insight, whereas in other cases they will be taken following further iteration.

Other properties are mentioned, but in each case they can more sensibly be represented as an attribute of an object. For example, reservation number is an integer, a value of a basic data type.

We will regard things like bills and invoices as separate classes, outside the system, because they have lifetimes very different from those of reservations and the like. The lifetime of a bill may even exceed the lifetime of an individual hotel. This situation comes from the fact that bills are considered 'legal documents' from the accounting perspective, and thus fall under the regulation-mandated storage rules. As such, they will need to be handled by a separate accounting system.

Our classes are now:

Guest, Room, Hotel, HotelChain, Reservation, RoomType.

We now add an entry for *RoomType* to the glossary, as shown in Table 6.

Table 6 Adding *RoomType* to the glossary

Term	Category	Definition
<i>RoomType</i>	Concept	The cost of a type of <i>Room</i> , the number of guests it can accommodate, and the kinds of bed (single, double, twin) that such a <i>Room</i> has.

SAQ 2

Identify the associations needed to support the use cases of our first iteration.

ANSWER.....

We need to consider each class against every other class. For *Guest*, we consider relationships with each of the other five classes; then for *Room*, we need to consider only four other classes, as the *Guest–Room* pair has already been considered. Note that in the following, though we identify each relationship only once, this does not imply directionality.

Guest: A *Guest* can occupy a *Room*, be a customer of the *HotelChain*, and have *Reservations*.

Room: A *Room* belongs to a *Hotel*, can be allocated to *Reservations*, and is of a *RoomType*.

Hotel: A *Hotel* belongs to a *HotelChain*, has *Reservations*, and has *RoomTypes*.

HotelChain: See above.

Reservation: A *Reservation* is for a *RoomType*.

RoomType: See above.

We add entries to our glossary corresponding to the associations found in SAQ 2, as shown in Table 7.

Table 7 Adding associations to the glossary

Term	Category	Definition
<i>Guest–Room</i>	Association	A <i>Guest</i> can occupy a <i>Room</i> .
<i>Guest–HotelChain</i>	Association	A <i>Guest</i> can be a customer of the <i>HotelChain</i> .
<i>Guest–Reservation</i>	Association	A <i>Guest</i> can have a number of <i>Reservations</i> . These include current <i>Reservations</i> , including the period when the <i>Guest</i> is staying in the <i>Hotel</i> , and past <i>Reservations</i> , which are recorded for reasons outside the current domain analysis.
<i>Room–Hotel</i>	Association	A <i>Room</i> belongs to a <i>Hotel</i> .
<i>Room–Reservation</i>	Association	A <i>Room</i> can be subject to a <i>Reservation</i> . A <i>Room</i> is subject to a <i>Reservation</i> from the time a <i>Guest</i> reserves it to the time that the <i>Guest</i> checks out.
<i>Room–RoomType</i>	Association	A <i>Room</i> is of a given <i>RoomType</i> .
<i>Hotel–HotelChain</i>	Association	A <i>Hotel</i> belongs to the <i>HotelChain</i> .
<i>Hotel–Reservation</i>	Association	A <i>Hotel</i> has a number of <i>Reservations</i> . These include current <i>Reservations</i> , including the period when the <i>Guest</i> is staying in the <i>Hotel</i> , and past <i>Reservations</i> , which are recorded for reasons outside the current domain analysis.
<i>Hotel–RoomType</i>	Association	A <i>Hotel</i> contains a number of <i>RoomTypes</i> .
<i>Reservation–RoomType</i>	Association	A <i>Reservation</i> is for a given <i>RoomType</i> .

SAQ 3

Draw the initial class diagram for the domain model.

ANSWER.....

A concept diagram corresponding to our analysis so far is shown in Figure 3.

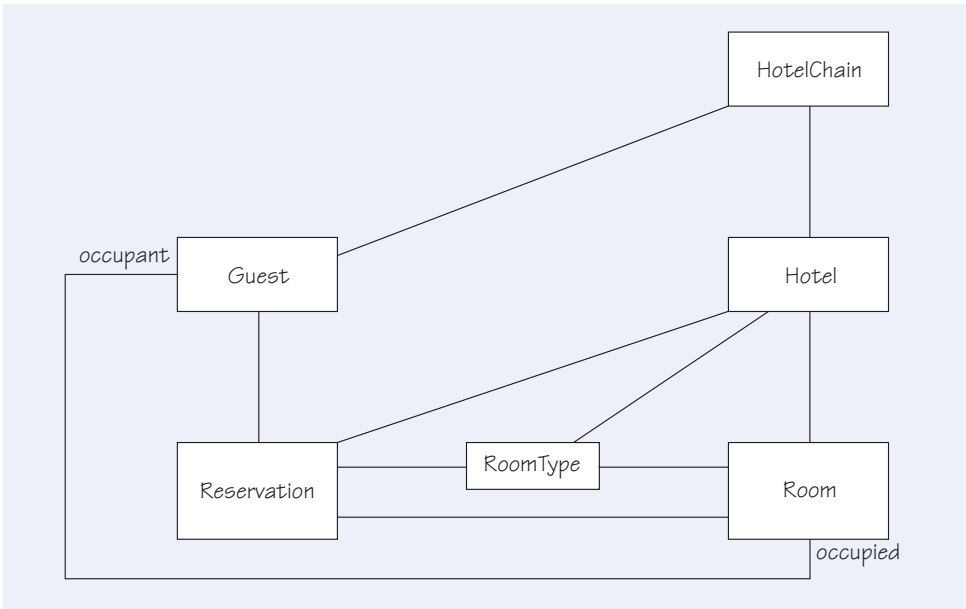


Figure 3 Concept diagram for the hotel system

We have also chosen to add a couple of role names, but this was not requested in the question. For the other associations, although we have quite a rich diagram, there are no pairs of classes that have more than one association directly between them, and the associations are mostly fairly obvious. In any case, they are identified in the glossary.

SAQ 4

- (a) Go through each step of each of the use cases for this iteration, checking that the relevant associations are present in the class diagram.
- (b) Add multiplicities to the diagram in Figure 3.

ANSWER.....

- (a) The relevant associations are indeed present.
- (b) The concept diagram in Figure 4 shows the multiplicities for each of the associations.

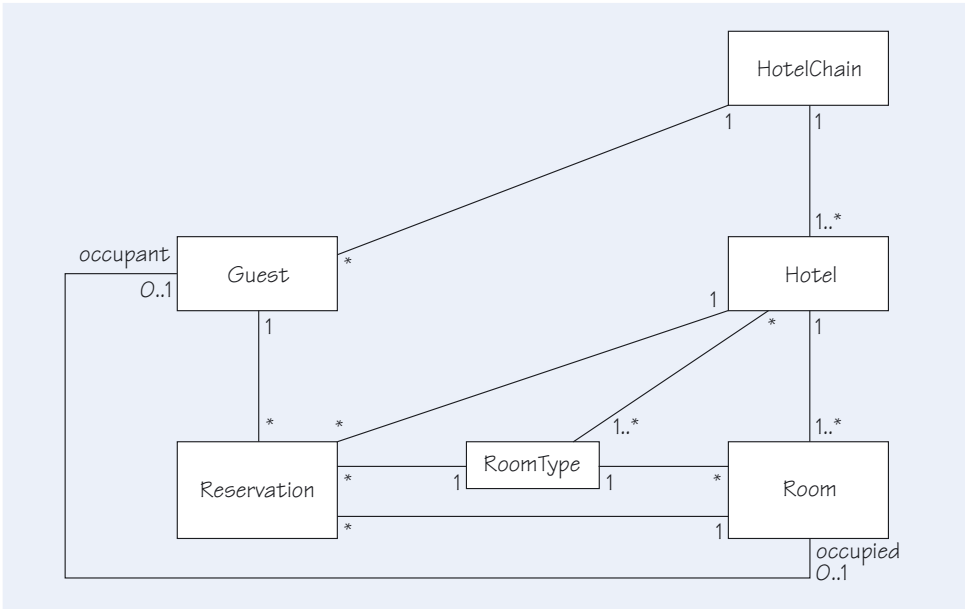


Figure 4 Concept diagram with multiplicities

SAQ 5

By working through the use cases for the current iteration step by step, add the attributes needed at this stage to the concept diagram in Figure 4, together with their corresponding data types.

Note that in analysis, there are some basic data types, such as Number, String, Character, Date and Boolean; where none of these seems appropriate we use a **placeholder**, a made-up name such as *RoomKind*, employed just for the time being, until we have sufficient information to decide what actual data type to use.

ANSWER.....

The *make reservation* use case (UC1) suggests several attributes, as shown in Table 8.

Table 8 Attributes suggested by the *make reservation* use case

Step	Justification for adding attributes
2	Refers to selection of a desired hotel, presumably chosen by name: add the attribute <i>name</i> to <i>Hotel</i> .
6	Refers to identification, contact details and credit card details: add the attributes <i>name</i> , <i>address</i> , <i>creditCardDetails</i> and <i>id</i> (identity) to <i>Guest</i> .
7	Refers to a reservation number: add the attribute <i>number</i> to <i>Reservation</i> . In order to create a <i>Reservation</i> , we need to add <i>startDate</i> , <i>endDate</i> and <i>reservationDate</i> to <i>Reservation</i> . We also need to know the room number: add the attribute <i>number</i> to <i>Room</i> . And we need to know the room type: add the attributes <i>kind</i> and <i>cost</i> to <i>RoomType</i> .

The other use cases suggested no attributes to us. The types that we have allocated to each attribute in the revised concept diagram of Figure 5 are just placeholders at this point in the development.

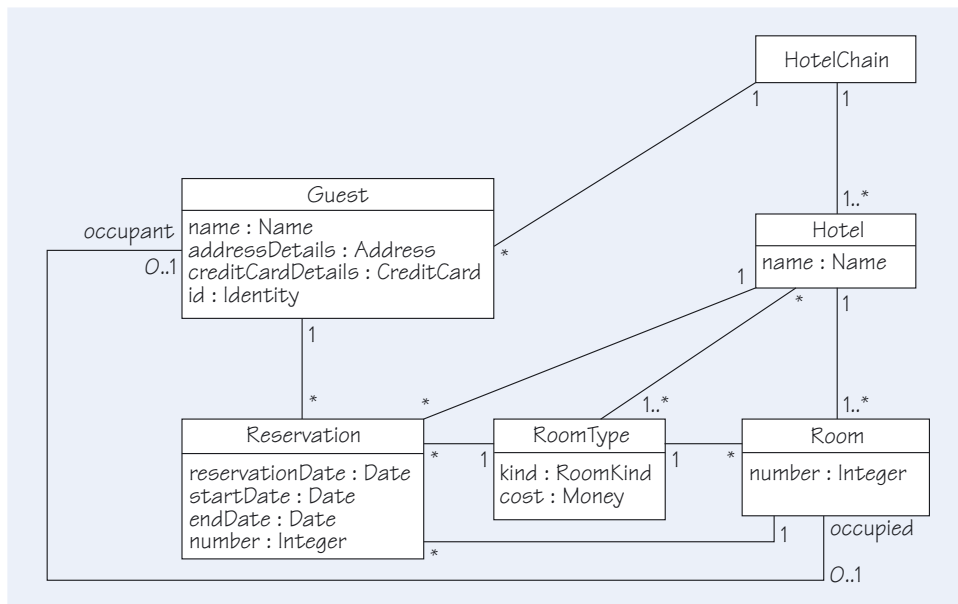


Figure 5 Concept diagram with attributes

We add the attributes identified in SAQ 5 to our glossary, as shown in Table 9. Further information will be added about the types later in the development, when they become more than placeholders.

Table 9 Adding attributes to the glossary

Term	Category	Definition
<i>name</i>	Attribute of <i>Hotel</i>	The name of a <i>Hotel</i> .
<i>name</i>	Attribute of <i>Guest</i>	The full name of a <i>Guest</i> .
<i>address</i>	Attribute of <i>Guest</i>	The address of a <i>Guest</i> .
<i>creditCardDetails</i>	Attribute of <i>Guest</i>	The credit card details of a <i>Guest</i> , sufficient to make a charge.
<i>id</i>	Attribute of <i>Guest</i>	A unique identifier for a <i>Guest</i> .
<i>number</i>	Attribute of <i>Reservation</i>	A unique identifier of a <i>Reservation</i> .
<i>startDate</i>	Attribute of <i>Reservation</i>	The date on which a <i>Reservation</i> starts.
<i>endDate</i>	Attribute of <i>Reservation</i>	The date on which a <i>Reservation</i> ends (the last night).
<i>reservationDate</i>	Attribute of <i>Reservation</i>	The date on which a <i>Reservation</i> was made.
<i>number</i>	Attribute of <i>Room</i>	The unique number identifying a <i>Room</i> in a <i>Hotel</i> .
<i>kind</i>	Attribute of <i>RoomType</i>	The details of a <i>Room</i> , in terms of the number of people it can accommodate and the sorts of bed it has (for example, <i>double</i> , <i>twin</i> , <i>single</i> , <i>familyOf5</i> , <i>familyOf4</i> , ...).
<i>cost</i>	Attribute of <i>RoomType</i>	How much the <i>RoomType</i> costs per night. Note that this is a simplification, as a <i>RoomType</i> may have different costs for different times of the year.

2.2 Summary of section

In this section you have seen how the concepts and associations in the **domain model** were selected. You also saw how attributes were identified based on the initial problem statement and the requirements from *Unit 4*.

3

Analysis

In analysis, we take our concept diagram produced during domain modelling, and add various pieces of information. This includes constraints on the model beyond those implied by multiplicities, **derived attributes** and so on. We can also revise our model as we think about its suitability for the task in hand. Furthermore, we may add classes, including association classes, to the same end. The end result will be to produce an analysis class diagram. We begin analysis by considering constraints on our concept model. Our analysis will then switch to a consideration of the system operations.

3.1 Constraints

We will identify the constraints on the analysis model by considering each of the six classes in turn, looking for association loops in which they participate. **Association loop invariants** can be expressed from any class context which they touch; so there is a degree of choice in how they are expressed.

Constraints for *HotelChain*

The *HotelChain* class has only one object; so no interesting constraints occur because of loops, and no other constraints suggest themselves.

Constraints for *Hotel*

We consider the *Hotel* class in relation to the four other classes not yet considered.

The *Guests* directly associated with a *Hotel* should include the *Guests* for whom we have *Reservations*. We use **includes** here rather than **=**, as once a *Hotel* knows about a *Guest*, we want to maintain that information regardless of any reservations. In OCL:

context *Hotel* **inv**:

`self.guest —> includes (self.reservation.guest)`

The collection of *Rooms* in a *Hotel* is the same as the collection obtained by pooling the *Rooms* for all the *RoomType* objects linked to the *Hotel* concerned. In OCL:

context *Hotel* **inv**:

`-- the Rooms of a Hotel are the Rooms of the RoomTypes of the Hotel.`

`self.room = self.roomType.room`

In fact, given the current multiplicities in the diagram, multiple *Hotels* can be associated with a given *RoomType*. This can be seen by the * multiplicity at the *Hotel* end of the *RoomType*–*Hotel* association. The original thinking here was that *RoomTypes* are shared across *Hotels*, but this leads to the situation where finding a *Hotel's Rooms* via its *RoomTypes* can result in our including *Rooms* from other *Hotels*. We will address this anomaly shortly when we review the concept diagram.

Constraints for *Room*

We have already said something about the associations between *Room* and *Hotel* and *RoomType*. The other remaining important constraints are probably more intuitively understood from the perspective of the *Reservation* and *Guest* concepts.

Remember that `self.guest` is a set etc.

Note the syntax for the **includes** operation etc.

Constraints for *Reservation*

The *Rooms* of the *RoomTypes* included in *Reservations* include the *Rooms* mentioned in *Reservations*:

```
context Reservation inv:
    self.roomType.room —> includes (self.room)
```

The *RoomTypes* of the *Hotel* include those of the *Reservations*:

```
context Reservation inv:
    self.hotel.roomType —> includes (self.roomType)
```

Reservation also has a constraint on its attributes:

```
context Reservation inv:
    self.endDate > self.startDate > self.reservationDate
```

Constraints for *Guest*

If a *Guest* occupies a *Room* in a *Hotel*, the collection of *Rooms* in the *Hotel* includes the *Room* the *Guest* occupies. However there might not be any *Rooms* occupied by the *Guest*:

```
context Guest inv:
    self.room —> notEmpty() implies self.hotel.room —> includes (self.room)
```

Constraints for *RoomType*

Hopefully, we have captured the important constraints on *RoomType* in the constraints above that mention this concept.



Exercise 1

In the above concept diagrams, we have conflated the concepts of guest, reservation-maker and bill-payer. Is this realistic? Would the people playing these roles necessarily be the same person?

Solution

There are a number of problems with the simplification identified. In reality, a hotel should allow for the roles identified to be distinct. It should be possible to have guests recorded who neither pay a bill nor make a reservation. For example, partners and children may be staying in the hotel along with a person who reserves and/or pays the bill. In fact, the hotel would almost certainly want a complete picture of who is staying in the hotel for a range of reasons.

A separate issue is whether it should be possible for one person to reserve on behalf of another, or pay on behalf of another. Allowing this gives more flexibility, but on the other hand having the credit card details of an actual occupier might make the sanction of charging for things like damage stronger. There is also a modelling issue here, since some people reserving rooms will also be guests, and some not, and not all guests will make reservations – for example, partners as above. You might like to try drawing a simple class diagram to see the problems this poses for inheritance.

For the reasons identified in Exercise 1, we want to separate out the role of a guest from those of people making reservations or paying bills.

Exercise 2

Are objects of the class *RoomType* shared across hotels? Does this present any problems?



Solution

The model permits the sharing of *RoomType* objects across hotels. This can be seen by the multiplicity at the *Hotel* end of the relevant association. However, this is problematic: if we try to find all of the rooms a hotel has by navigating via the *RoomType* objects, we may include rooms from other hotels! In any case, there is an argument for making the *RoomType* objects associated with a *Hotel* unique to that hotel, as prices for a given type of room are likely to vary depending on location and local market conditions. And in fact, this is what we decide to do.

Exercise 3

Does the model cover reservations for multiple rooms?



Solution

No. This can be seen from the multiplicities of 1 on the *Reservation–RoomType* and *Reservation–Room* associations.

The answers we came up with to the three exercises above suggest that it would be appropriate to adjust our model. This iterative approach is to be expected in a real development, as stakeholders should be consulted when analysis produces artefacts that we need to check against reality. We will also add an association class to allow us to record how many *Rooms* of a given *RoomType* are included in a given reservation. Our analysis class diagram is shown in Figure 6.

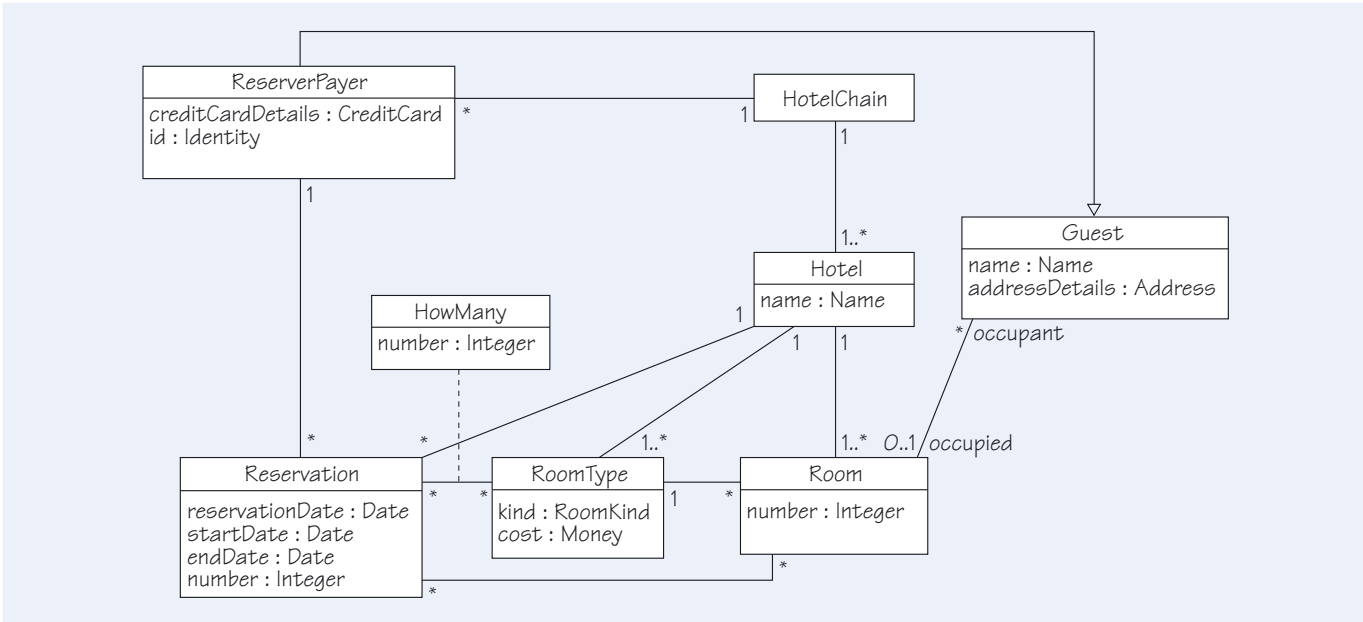


Figure 6 Analysis class diagram

Now we need to update the glossary. We need to remove the current entry for *Guest*, replacing it with two new entries, for *Guest* and *ReserverPayer*, and add an entry for the association class, *HowMany*. See Table 10.

Table 10 Updating concepts in the glossary

Term	Category	Definition
<i>Guest</i>	Concept	Somebody who might stay in a <i>Hotel</i> .
<i>ReserverPayer</i>	Concept	A <i>Guest</i> who reserves and/or pays for a <i>Room</i> .
<i>HowMany</i>	Concept	An association class to record the number of <i>Rooms</i> of a given <i>RoomType</i> in a <i>Reservation</i> .

We also need to remove the existing associations involving *Guest*, and represent the associations between *ReserverPayer*, *Reservation* and *HotelChain*, and the association between *Guest* and *Room*. See Table 11.

Table 11 Updating associations in the glossary

Term	Category	Definition
<i>Guest–Room</i>	Association	A <i>Guest</i> can occupy a <i>Room</i> .
<i>ReserverPayer–HotelChain</i>	Association	A <i>ReserverPayer</i> can be a customer of the <i>HotelChain</i> .
<i>ReserverPayer–Reservation</i>	Association	A <i>ReserverPayer</i> can have a number of <i>Reservations</i> . These include current <i>Reservations</i> , including the period when the <i>Guest</i> is staying in the <i>Hotel</i> , and past <i>Reservations</i> , which are recorded for reasons outside the current domain analysis.

Finally, we need to update the attribute entries to show how some of the attributes in *Guest* have been reallocated to the *ReserverPayer* class. We also record the attribute of the *HowMany* association class. See Table 12.

Table 12 Updating attributes in the glossary

Term	Category	Definition
<i>number</i>	Attribute of <i>HowMany</i>	The number of <i>Rooms</i> of a given <i>RoomType</i> in a <i>Reservation</i> .
<i>name</i>	Attribute of <i>Guest</i>	The full name of a <i>Guest</i> .
<i>address</i>	Attribute of <i>Guest</i>	The address of a <i>Guest</i> .
<i>creditCardDetails</i>	Attribute of <i>ReserverPayer</i>	The credit card details of a <i>ReserverPayer</i> , sufficient to make a charge.
<i>id</i>	Attribute of <i>ReserverPayer</i>	The identity of a <i>ReserverPayer</i> .

3.2 System operations

Now that we have the structural model, we can describe the effects of the use cases and detailed requirements. The main operations of the system can be identified from

the use cases. We start by defining the system operations, which describe the behaviour of the system considered as a black box. These operations define the external interface of the system – what any implementation will have to provide to the user, independently of how it is provided. (An implementation will probably provide a user interface for the interactions that take place in the scenarios of a corresponding use case.) An acceptable implementation is one that satisfies the specification of the system operations. Ultimately, system operations can be used to construct tests of the system, as discussed in a later unit.

Use cases give an overview of the behaviour of the system; this may be enough in some cases. System operations add detail, and will be most useful for more complex use cases. For each use case, we have already defined a precondition and a postcondition; now that we also have a structural model, we can start detailing these as pre- and postconditions of operations, and specifying them in terms of the structure of the system. Initially, each use case will correspond to an operation for the whole system.

We will write specifications of operations in English, adopting the conventions that a precondition is described in the present tense, and a postcondition is described in the future tense. The typical syntax for an operation specification is as follows.

```
context TypeName::operationName(parameter1 : Type1, ...) : ReturnType
  pre:
    -- condition
  post:
    -- condition
```

An operation is defined within a *context* (*TypeName* is the class in which the operation is defined), and with a *name*, a series of *parameters* and their types, and the type of the *returned value* (when there is a returned value); this is the operation's **signature**.

The precondition is a constraint on the parameters of the operation and on the properties (values of attributes and links) of the context; the precondition has to be true before the operation is executed. The postcondition is a constraint on the state after the operation is executed, in relation to the state before the operation was executed; these states are represented by the properties of the context and the returned object (if any).

We will specify system operations for three of our use cases, omitting those for *check out guest*. Part of our rationale is that tackling the first three use cases is sufficient both to illustrate the overall approach and to establish the architecture of the system. For the same reasons, we will not address all of the use cases to the same level of detail.

3.3 Specifying operations

As for use cases, there is no fixed way of deciding the level of granularity at which system operations should be defined. The main criterion is that they should be useful in understanding the effects of a business process on the system. We opt, initially, to define a single system operation for each use case corresponding to a main business process. Breaking the system operation into a set of more detailed operations will be done afterwards, when we make decisions about allocation of responsibilities to classes. At this point in the design, it is convenient to allocate all the identified system operations to the **system class**. The system class in this example is *HotelChain*, which represents the whole system and has a single instance; it will coordinate the behaviour of other objects in the system in order to carry out the operations. A system class is just a convenient device to help us to make progress with the system design. However, introducing such a class at this stage does not mean that it will still be present in the final system design, or that other classes will not be introduced to orchestrate part of the system behaviour. In fact, in large systems, the responsibility of this class is

typically distributed among a number of separate classes. At this point, however, it is a useful abstraction: a single point in the system from which all behaviours can be orchestrated, and a single point of entry for all interactions of the system with the environment. The system class has one defining characteristic: there is only one instance of such a class. The system class keeps track of objects in the system via associations – either directly, as is the case for the class *Hotel*, or indirectly, through the associations that this class has with other classes in the system.

Table 13 below reminds us of the description of the *make reservation* use case, as given in *Unit 4*.

Table 13 The *makeReservation* system operation

Identifier and name	UC1 <i>make reservation</i>
Initiator	<i>Reserver or Receptionist</i>
Goal	A room in a hotel is reserved for a guest.
Precondition	None (that is, there are no conditions to be satisfied prior to carrying out this use case).
Postcondition	A room of the desired type will have been reserved for the guest for the requested period, and the room will no longer be free for that period.
Assumptions	The expected initiator is a reserver or receptionist, using a web browser to perform the use case. The guest is not already known to the hotel's software system (see step 5).
Main success scenario	
1	The reserver/receptionist makes a reservation request.
2	The reserver/receptionist selects the desired hotel, dates and type of room.
3	The hotel system provides the availability and price for the request; that is, one or more offers are made.
4	The reserver/receptionist accepts the offer.
5	The hotel system requests identification and contact details.
6	The reserver/receptionist provides identification and contact details of the guest for the hotel system's records, as well as credit card details.
7	The hotel system creates a reservation and gives it a number.
8	The hotel system allocates a room to the reservation.
9	The hotel system reveals the reservation number to the reserver/receptionist.
Extensions	
3.a.1	<i>No availability in this hotel.</i> The hotel system offers alternatives at a different hotel or hotels within a limited radius.
3.b.1	<i>No availability in this hotel or in any other hotels within a limited radius.</i> The hotel system informs the reserver and terminates the use case.
4.a	<i>Offer not accepted.</i> The hotel system terminates the use case.
6.a	<i>Guest already on record.</i> The scenario continues at step 7.

In order to specify the corresponding system operation, we need to look at the class diagram as defined in Figure 6. Note that although the class diagram allows for more than one room type for each reservation, the use case described above deals only with

one reservation, and that reservation deals with only one room type. To keep it simple, we will specify a system operation that corresponds to the use case as defined above.

An operation to achieve the goal of this use case will need to have as its inputs the *Hotel*, the start and end dates, the *RoomType*, and the *ReserverPayer* responsible for the reservation and its payment.

SAQ 6

Consider the *make reservation* use case described above, and write the signature for a corresponding operation.

ANSWER.....

context HotelChain::makeReservation(hotel : Hotel, startDate : Date, endDate : Date, roomType : RoomType, reserverPayer : ReserverPayer)

We should now define the pre- and postconditions for the corresponding operation. To help you specify them, let us look at two snapshots, illustrating the before and after states for the execution of this operation. Figure 7 illustrates a sample state of the system before the *makeReservation* operation is executed.

From now on, we will use *roomType* to refer to an instance of the class *RoomType*, and so on.

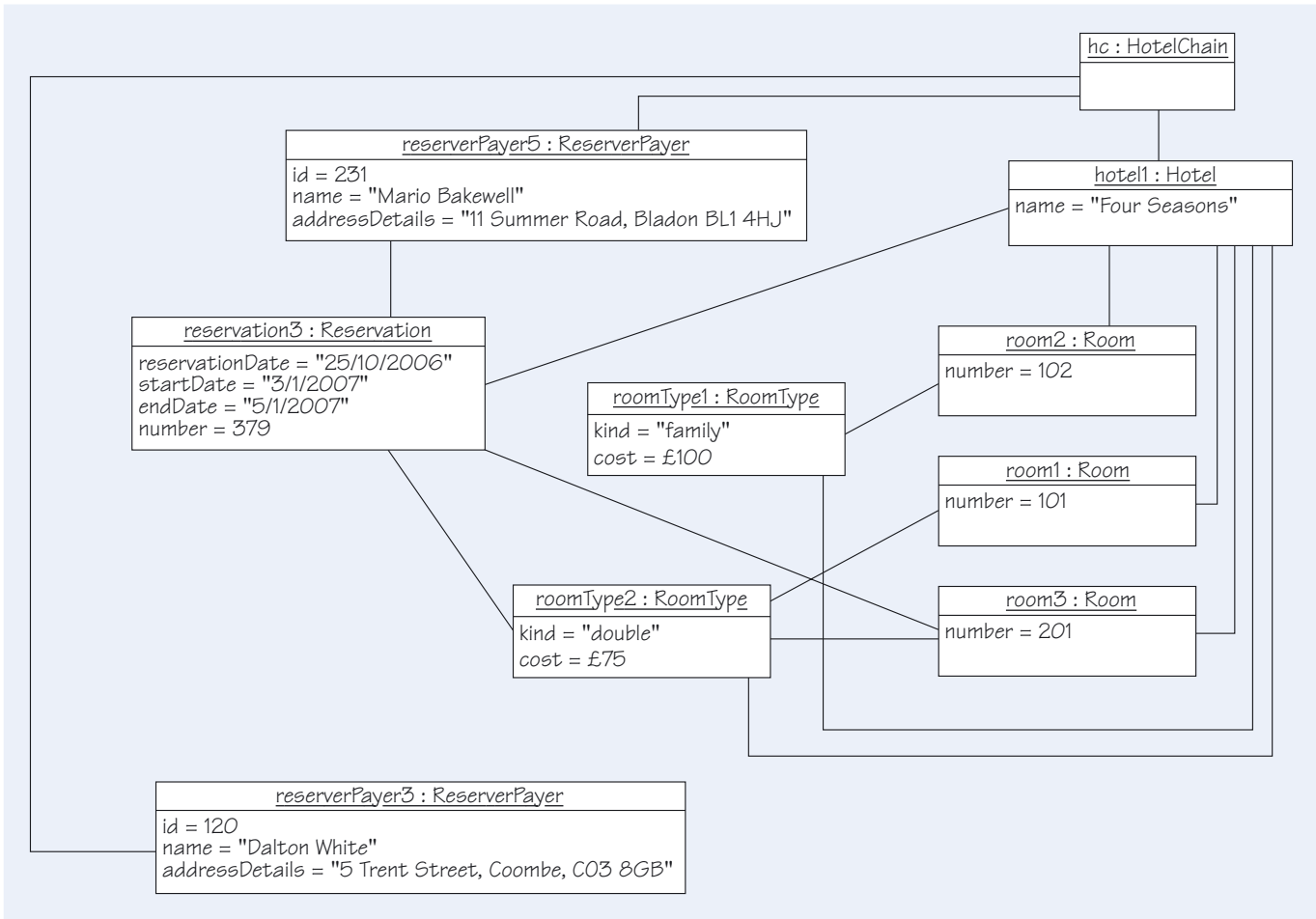


Figure 7 Snapshot illustrating a sample state of the system before the *makeReservation* operation is executed (credit card details have been omitted from *ReserverPayer* objects)

SAQ 7

Consider the snapshot in Figure 7, and draw a snapshot after the execution of the system operation *makeReservation*(*hotel1*, "8/4/2007", "10/4/2007", *roomType2*, *reserverPayer3*).

Assume that the allocated room is *room1* (with room number 101).

Figure 7 is available on the course website.

ANSWER.....

Our answer is shown in Figure 8.

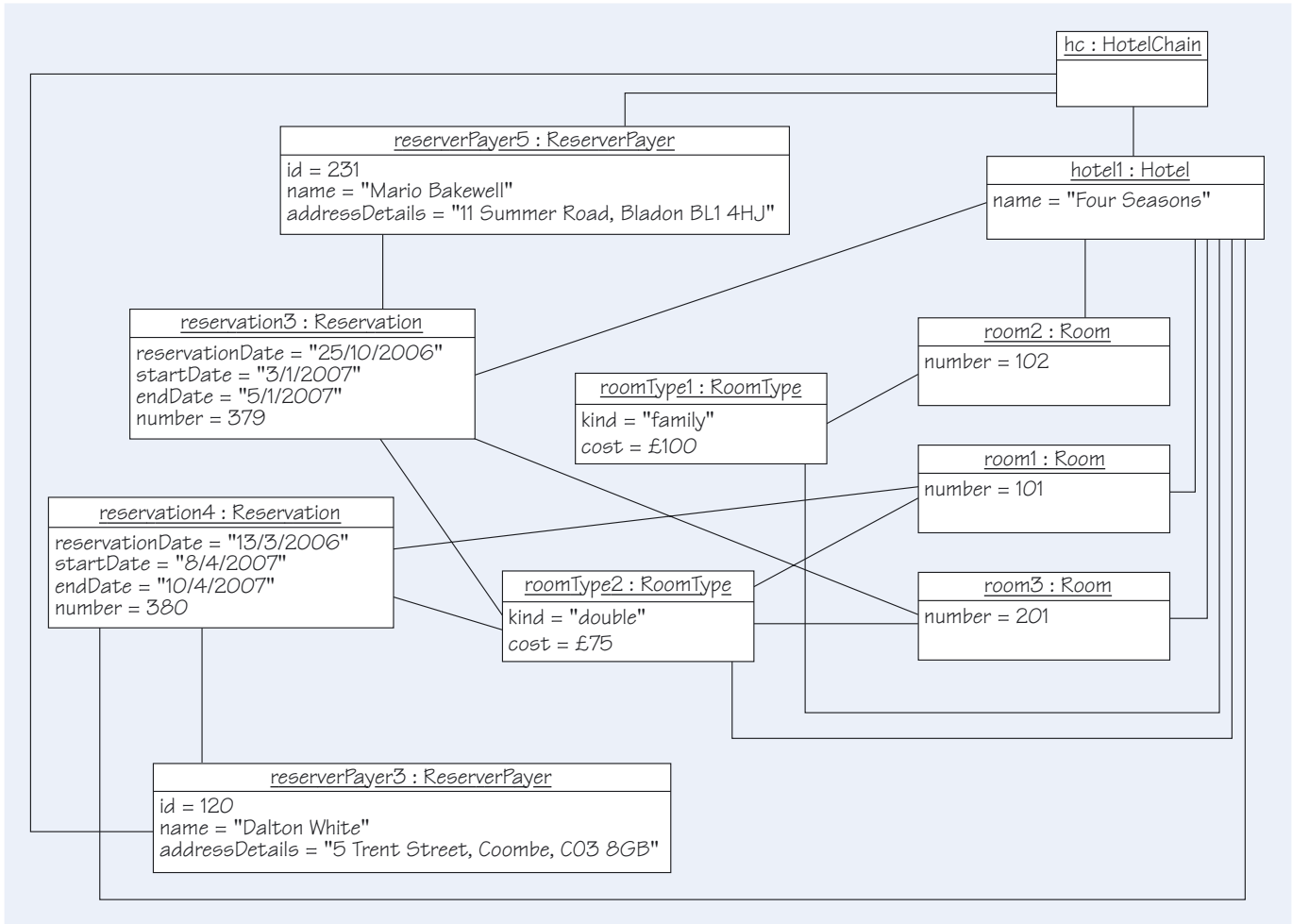


Figure 8 Snapshot illustrating the state after another reservation has been made: *makeReservation*(*hotel1*, "8/4/2007", "10/4/2007", *roomType2*, *reserverPayer3*)

The whole operation can now be specified by showing the pre- and postconditions. Let us look first at the preconditions.

context HotelChain::makeReservation(*hotel* : Hotel, *startDate* : Date, *endDate* : Date, *roomType* : RoomType, *reserverPayer* : ReserverPayer)
pre:
-- *startDate* is before *endDate*
-- *roomType* is one of *hotel's RoomTypes*

Note the convention (mentioned above) of writing the preconditions in the present tense. Note also that we have started to add more detail than we had in the use case, as we now have a class diagram with the initial representation of the software

elements, and we can see how these are affected by the use case. The model had a constraint about dates of a reservation, and we need to impose this so that the *makeReservation* operation complies with the model. Also, the hotel has to have the required room type. The structural model shows an association between *Hotel* and *RoomType*, but not all hotels have rooms of all types. Therefore, the precondition needs to stipulate that the hotel has rooms of the requested *roomType*.

We should now look at the snapshots before and after the operation is executed, as shown in Figures 7 and 8, and express the changes in the postcondition. We start by looking only at the changes that take effect when the main scenario of the use case is followed. The exception related to availability (we are addressing availability in only one hotel) will be dealt with later. In writing the signature of the system operation with a parameter *reserverPayer*, we assume that at some point in the interaction either a new *reserverPayer* was created (as assumed in the main scenario, for example in a login interaction), or an existing *reserverPayer* was retrieved (as in exception 6.a of the use case).

From now on, for the rest of this unit, we assume there is an object called *hotelChain* representing the Hotel Chain.

context *HotelChain::makeReservation*(*hotel* : *Hotel*, *startDate* : *Date*, *endDate* : *Date*, *roomType* : *RoomType*, *reserverPayer* : *ReserverPayer*)

post:

- a new *Reservation* object, *reservation*, will have been created (object creation)
- *hotelChain* will be linked to *reserverPayer*
- *hotel* will be linked to *reservation*
- *reserverPayer* will be linked to *reservation*
- *reservation* will be linked to *roomType*
- *reservation* will be linked to a previously free *Room*

We need now to consider how to deal with the exceptions related to the availability of a room, and to the guest being known to the hotel.

Room availability

At this stage in analysis, it is inappropriate to consider the details of how availability of the requested room type will be represented (for example, as an attribute of *Room*, or by derivation from the reservations). However, we do need to decide who has the responsibility for recording the information about it. This is because in analysis we allocate responsibilities, but we do not prescribe how they will be met. For the moment, let us assume that each hotel will know about the availability of a room of a specific type. We represent this by an operation assigned to *Hotel*: given a room type and a range of dates, it will determine the availability of a room of that type. At this point, we are not dealing with offers in other hotels, or offers of rooms of other types.

SAQ 8

Discuss how you would apply GRASP to assign *Hotel* the responsibility for finding out the availability of a room of a specified type.

ANSWER.....

GRASP Expert would apply here, in assigning the responsibility to the *Hotel* class, as *Hotel* knows about all its rooms, room types and reservations.

SAQ 9

Consider the operation that would determine the availability in a hotel of a room of a specific type, for given start and end dates, and write its signature.

ANSWER.....

context Hotel::available(startDate : Date, endDate : Date, roomType : RoomType) : Boolean

A reservation in a hotel can be made only if availability is confirmed. We therefore change the signature of *makeReservation* to return a string indicating whether a reservation has been made, and if so, containing the reservation number.

context HotelChain::makeReservation(hotel : Hotel, startDate : Date, endDate : Date, roomType : RoomType, reserverPayer : ReserverPayer) : String

Completing the *makeReservation* operation

Now that we have addressed exceptions in the use case, we can complete the specification of the *makeReservation* operation as follows.

context HotelChain::makeReservation(hotel : Hotel, startDate : Date, endDate : Date, roomType : RoomType, reserverPayer : ReserverPayer) : String

pre:

- startDate is before endDate
- roomType is one of hotel's roomTypes

post:

- if available(startDate, endDate, roomType) executed in hotel returned true then
- a new Reservation object, reservation, will have been created (object creation)
- hotelChain will be linked to reserverPayer
- hotel will be linked to reservation
- reserverPayer will be linked to reservation
- reservation will be linked to roomType
- reservation will be linked to a previously free Room
- a string indicating the reservation number will have been returned
- otherwise, a string indicating failure will have been returned

Recall the description of the *cancelReservation* use case from Unit 4.

Table 14 The *cancelReservation* system operation

Identifier and name	UC2 <i>cancel reservation</i>
Initiator	<i>Receptionist or Reserver</i>
Goal	A guest's reservation of a room at a hotel is cancelled.
Precondition	There is a reservation for a room at a hotel for a guest with a correct reservation number.
Postcondition	The reservation will have been cancelled, the allocated room will be freed, and a charge to the credit card will have been made if the cancellation was initiated within 24 hours of the start of the reservation.
Assumptions	The expected initiator is a receptionist or reserver.
Main success scenario	
1	The receptionist/reserver requests the cancellation of a reservation.
2	The receptionist/reserver enters the reservation number or the dates and name and address.
3	The hotel system retrieves the corresponding reservation.
4	The hotel system frees the allocated room.
5	The receptionist/reserver confirms the cancellation.
6	If the cancellation was initiated within 24 hours of the beginning of the reservation, the hotel system charges the credit card whose details are kept in the reservation.
7	The reservation is cancelled, and the hotel system informs the receptionist/reserver of the cancellation.
Extensions	
None	

Exercise 4

Consider the *cancel reservation* use case described above.

- Assume the situation as given in the snapshot in Figure 8, and draw a snapshot of the situation after the cancellation of the reservation with reservation number 379.
- Write the signature of the corresponding system operation *cancelReservation*.
- Write its pre- and postconditions.

Solution

- Our post-cancellation snapshot is shown in Figure 9.



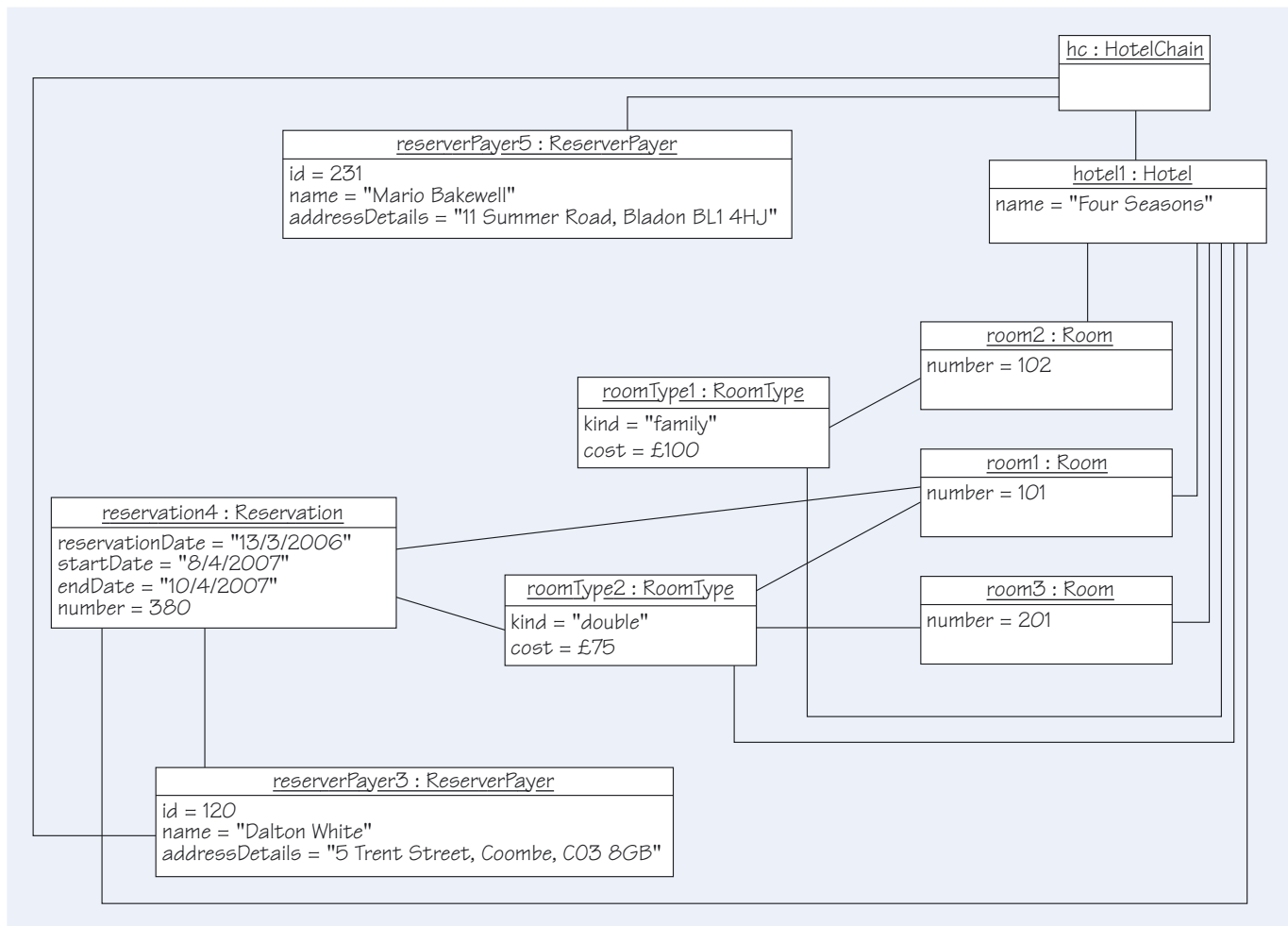


Figure 9 Snapshot illustrating a sample state after *cancelReservation* is executed

(b) The signature for *cancelReservation* is as follows.

context HotelChain::cancelReservation(hotel : Hotel, reservation : Reservation)

(c) The specification of pre- and postconditions is as follows.

context HotelChain::cancelReservation(hotel : Hotel, reservation : Reservation)

pre:

-- reservation is linked to hotel and reservation.startDate is not in the past

post:

-- reservation will no longer be linked to hotel, to the reserverPayer who made it,

-- nor to the allocated room or roomType

For this iteration, when a reservation is cancelled, the *ReserverPayer* that made the reservation is still kept. It is useful to have their details if they book again, and they may be a regular customer who will not want the inconvenience of supplying their details repeatedly.

Recall the description of the *check in guest* use case from Unit 4.

Table 15 The *checkInGuest* system operation

Identifier and name	UC3 <i>check in guest</i>
Initiator	<i>Receptionist</i>
Goal	A guest takes up a reservation and occupies a room at the desired hotel.
Precondition	There is a room (of the desired type) reserved for the correct reservation number, and the guest can pay for the room.
Postcondition	The guest will have been allocated to a room for the period identified in the reservation, and a bill will have been opened for the duration of the stay.
Assumptions	The guest is already known to the hotel system. The hotel is confident that the guest can pay. For example, the guest has a valid credit card.
Main success scenario	
1	The guest provides a reservation reference number, or the dates and name and address, to the receptionist.
2	The receptionist enters the reference number, or the dates and name and address, to find the reservation.
3	The hotel system retrieves the details of the requested reservation.
4	The receptionist confirms the details of the room type and duration of the stay with the guest.
5	The hotel system allocates the room to the guest.
6	The hotel system opens a bill for the guest. (It could be that there is a separate billing package, which must be notified upon check in.)
7	The receptionist issues a key to the guest.
Extensions	
None	

The use case as defined above allows for only one check in at a time. The class diagram treats *ReserverPayers* and other *Guests* differently; we will, therefore, look at the system operation for each case separately. We start by looking at a system operation that checks in a *ReserverPayer*.

Exercise 5

Consider the *check in guest* use case described above.

- Assume the situation as given in the snapshot in Figure 8, and draw a snapshot of the situation after the check in for the reservation with reservation number 379.
- Now assume that for the same reservation and allocated room, the reserver's partner is also to check in. Let us say that the partner is 'Charlie Watson' and lives at the same address. Draw a snapshot of the situation after execution of the check in.
- Write the signature of the corresponding system operation, *checkInGuest*.
- Write its pre- and postconditions.



Solution

(a) Our post-check in snapshot for a *ReserverPayer* is shown in Figure 10.

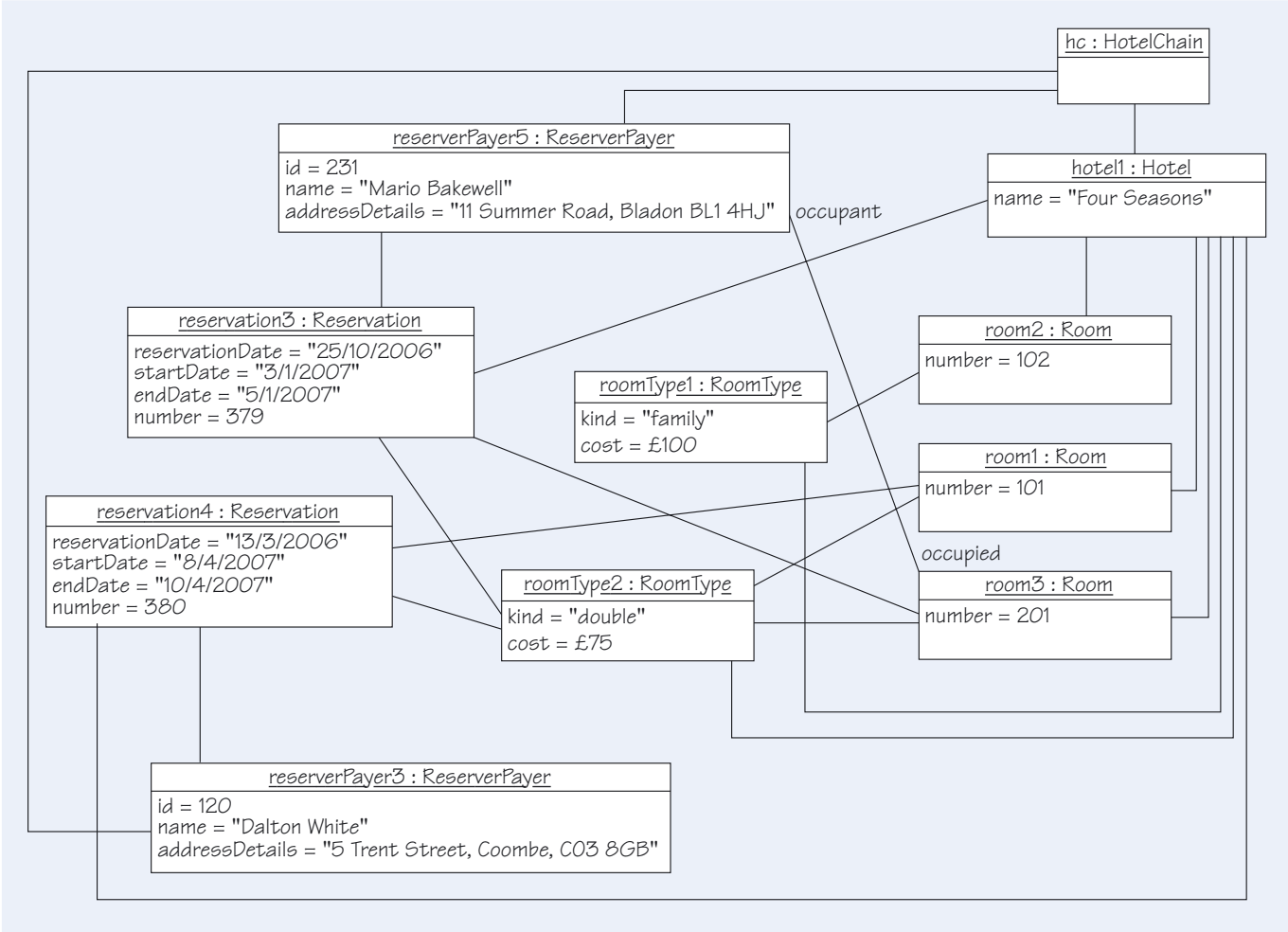


Figure 10 Snapshot illustrating a sample state of the system after a *ReserverPayer* checks in

(b) Our post-check in snapshot for another *Guest* is shown in Figure 11.

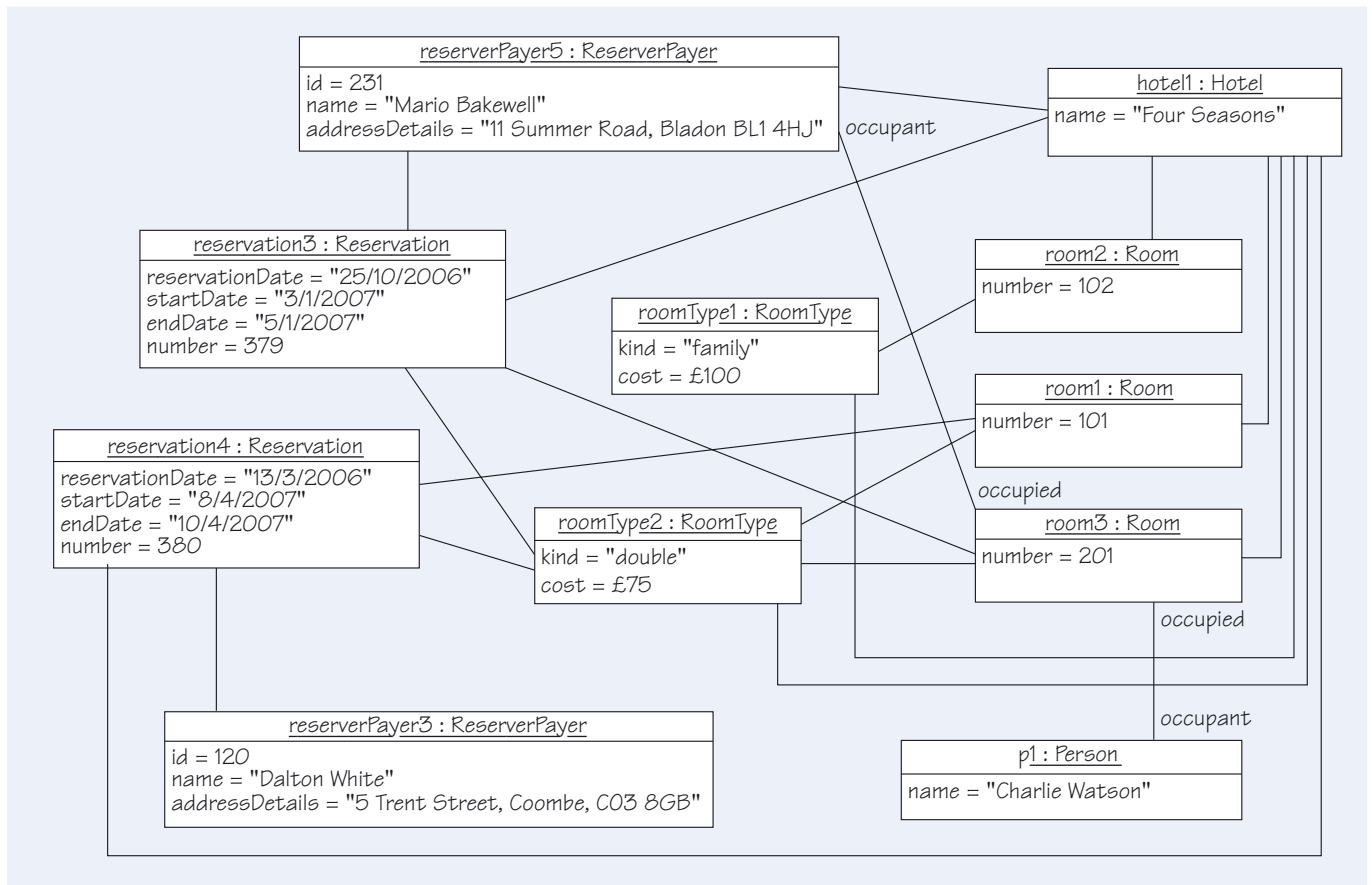


Figure 11 Snapshot illustrating a sample state of the system after another *Guest* checks in

(c) The signature for *checkInGuest* is as follows.

context HotelChain::checkInGuest(hotel : Hotel, reservation : Reservation, guest : Guest)

(d) The specification of pre- and postconditions is as follows.

context HotelChain::checkInGuest(hotel : Hotel, reservation : Reservation, guest : Guest)

pre:

-- reservation is linked to hotel

-- a Room is linked to reservation, and the Room is of the RoomType for reservation

post:

-- the Room linked to reservation will be linked to guest

3.4 Summary of section

In this section you have seen how to place constraints on the structural model. You also saw how to relate the specifications of system operations to the use cases, and to the effects on the structural model of the system.

4 Design – assigning responsibilities

In design, the class diagram defined during analysis is refined in a number of ways.

- ▶ More responsibilities are assigned to classes, by applying basic design principles and by analysing the pre- and postconditions of system operations.
- ▶ The navigation of associations is established, based on the way objects in the system need to collaborate to realise the system operations.
- ▶ Operations are added to the classes, based on the assigned responsibilities.

For the remainder of this unit, we will use the class diagram for the *HotelChain* introduced in Subsection 3.1 (see Figure 12).

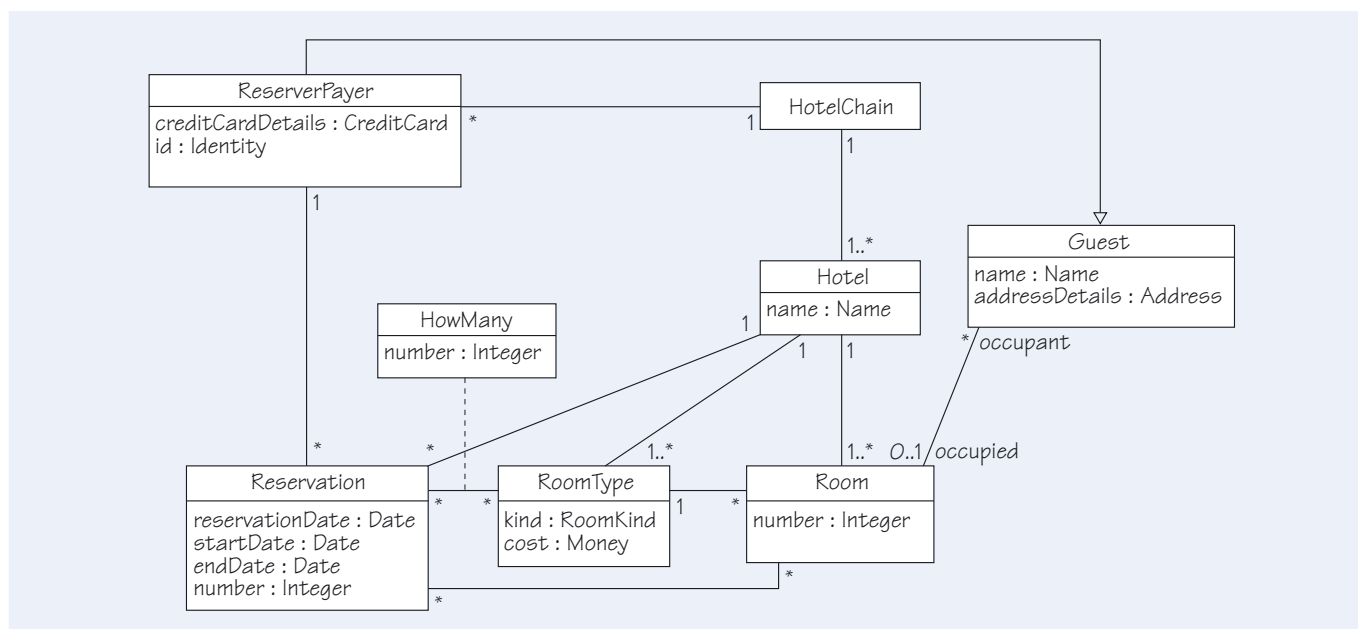


Figure 12 *HotelChain* class diagram

So far, we have assigned operations to the system class *HotelChain*. The system class has the responsibility of orchestrating the behaviour of the objects in the system and keeping track of them.

4.1 Value identities

So far, we have been distinguishing between objects without saying anything about how this is done: one hotel is distinguished from another because the hotels have different object identifiers. We need a more meaningful way to distinguish between objects, one that can be used in software. As you saw in *Unit 5*, many objects can be distinguished from each other by the values of one or more of their attributes: a *Reservation* by its reservation *number*, a *ReserverPayer* by its *id*, and so on. This form of identity is known as **value identity**. In *Unit 5*, we have seen how to represent value identities on a class diagram by using the UML notation of qualified association: in a qualified association, a qualifier explicitly indicates the set of object properties that represents the identity of objects at the opposite end of the association.

In Figure 13, we have redrawn the class diagram, showing value identities for various classes; hotels are distinguished by *name*, rooms in a hotel by *number*, reservations by *number*, and *ReserverPayers* by *id*.

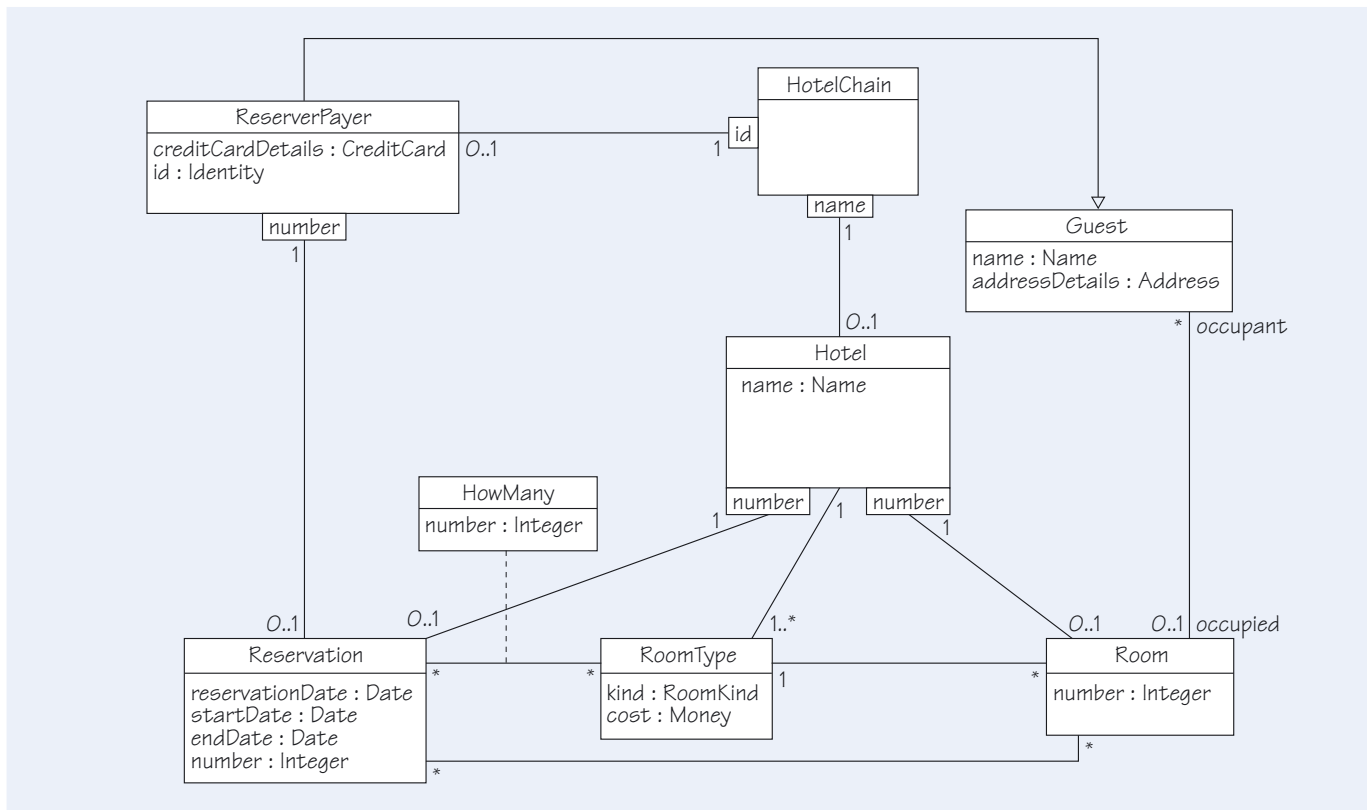


Figure 13 *HotelChain* class diagram with qualified associations

4.2 Verifying and fulfilling pre- and postconditions

In Subsection 3.3, we described system operations by giving their pre- and postconditions. The precondition states what has to be true before the execution of the operation; the postcondition states what has to be true after the execution of the operation. By using pre- and postconditions, we can rigorously describe the conditions under which the service provided by an operation can be used, and the result obtained after the service is provided. This captures the essential detail about *what* services are provided, without needing to state *how* they are going to be provided. Writing pre- and postconditions in a rigorous way also has the advantage of making us think about the requirements, clarifying any ambiguity, and rectifying any omissions. A rigorous postcondition is also the first step for verification and testing. A properly implemented system will successfully make the postcondition of an operation true whenever that operation is executed, provided that the matching precondition is satisfied.

Verifying preconditions

Preconditions are verified, and postconditions fulfilled, by objects interacting with each other in a predefined way that may result in objects, and links between those objects, being created or destroyed. To describe how these objects need to interact to verify and fulfil the pre- and postconditions of each operation, we must first identify the objects that are involved, and see how those objects and the links between them are affected.

Verifying preconditions may require consideration of:

- ▶ whether particular objects exist;
- ▶ whether particular links exist;
- ▶ the values of particular attributes.

Fulfilling postconditions may involve:

- ▶ creating particular objects;
- ▶ creating particular links;
- ▶ modifying the values of particular attributes;
- ▶ returning particular values.

Let us look at the *makeReservation* operation as specified in Subsection 3.3.

context *HotelChain::makeReservation*(*hotel* : *Hotel*, *startDate* : *Date*, *endDate* : *Date*, *roomType* : *RoomType*, *reserverPayer* : *ReserverPayer*) : *String*

pre:

- *startDate* is before *endDate*
- *roomType* is one of *hotel's* *roomTypes*

post:

- if *available*(*startDate*, *endDate*, *roomType*) executed in *hotel* returned true then
- a new *Reservation* object, *reservation*, will have been created (object creation)
- *hotelChain* will be linked to *reserverPayer*
- *hotel* will be linked to *reservation*
- *reserverPayer* will be linked to *reservation*
- *reservation* will be linked to *roomType*
- *reservation* will be linked to a previously free *Room*
- a string indicating the reservation number will have been returned
- otherwise, a string indicating failure will have been returned

We start by focusing on the precondition of this operation. In order to identify the parties involved in the collaboration to verify the precondition, we need to identify:

- ▶ who initiates the collaboration (this is the actor of the corresponding use case; in the case of *make reservation*, it is a *Reserver*);
- ▶ the object that orchestrates the collaboration (this is by definition the system object);
- ▶ other objects in the system with which the system object needs to collaborate in order to verify that the precondition of the operation is satisfied (to be able to identify these objects, we need to look at the operation's definition).

By looking at the arguments and the precondition of the operation, the objects involved are the *hotels*, *rooms* and *RoomType* objects in the system. The *hotels* are linked to the system object through a qualified association, capturing the fact that the system object can perform a mapping between the value identity, the *name* of the hotel in this case, and the identity of the corresponding object. The details of this mapping will be addressed at implementation, when features of the target programming language will be used to realise the mapping. The mapping need not be considered any further in design: all the information needed is already captured in the qualified association.

Similarly, *Room* objects are linked to *Hotel* objects by qualified associations with the qualifier *number*, and so on.

For now, we adopt the convention that verifying the precondition is initiated by a message between the actor that triggers the corresponding use case and the system object, and that the message name is given by the system operation name prefixed by 'can'. In the final system, the actor–system interaction will occur through a user interface, which we have not considered yet. Therefore, this message is only an abstraction of what will actually happen when the system is completed.

We will now access objects by their value identities where qualified associations have been added to the class diagram in Figure 13.

context HotelChain::canMakeReservation(hotelName : Name, startDate : Date, endDate : Date, roomType : RoomType) : Boolean

post:

- true will be returned if
- *startDate* is before *endDate*, and
- there is a *hotel* with *Name* *hotelName*, and it has rooms of *roomType*
- otherwise, false will be returned

Exercise 6

Consider the operation *cancelReservation* rewritten now with value identities.

context HotelChain::cancelReservation(hotelName : Name, reservationNumber : Number)

pre:

- the *Reservation* with *reservationNumber* is linked to the *Hotel* with *hotelName*
- the *start Date* for the *Reservation* with *reservationNumber* is not in the past

post:

- the *Reservation* with *reservationNumber* will no longer be linked to the *Hotel* with *hotelName*, to the *reserverPayer* who made it, nor to the allocated *room* or *roomType*

Define an operation to verify its precondition.

Solution

The following operation will verify the precondition of *cancelReservation*.

context HotelChain::canCancelReservation(hotelName : Name, reservationNumber : Number) : Boolean

post:

- true is returned if
- there is a *Hotel* with *hotelName*, and
- there is a reservation for that *Hotel* with number *reservationNumber*

The interactions required to process the *canMakeReservation* and *canCancelReservation* operations are sufficiently simple to be left until implementation.

Exercise 7

Consider the operation *checkInGuest* rewritten now with value identities. Note that check in is done guest by guest, and there is a slight difference between the way a *ReserverPayer* checks in and the way any other *Guest* checks in. *ReserverPayers* should already exist in the system, but information for other *Guests* is only recorded when they check in.

context HotelChain::checkInGuest(hotelName : Name, reservationNo : Integer, roomNo : Integer, name : Name, address : Address)

pre:

- the *Reservation* with *reservationNo* is linked to the *Hotel* with *hotelName*
- the *room* with *roomNo* is linked to the *Reservation*, and the *Room* is of the *RoomType* for the *Reservation*

```
post:
-- if name is not that of the ReserverPayer of the Reservation with reservationNo, then
--   a new Guest will be created, and its attributes set; this new Guest or,
-- otherwise, the ReserverPayer will be linked to the Room with roomNo
```

Define an operation to verify its precondition.

Solution
The following operation will verify the precondition of *checkInGuest*.

```
context HotelChain::canCheckInGuest(hotelName : Name, reservationNumber :
Number, roomNo : Integer) : Boolean
post:
-- true is returned if there is a Hotel with hotelName, and there is a Reservation with
-- reservationNo for that hotel, and the Room with roomNo is of the roomType for the
-- Reservation
```

Fulfilling postconditions

We can now consider the collaborations to satisfy the postcondition of a system operation. Once again, we will consider the *makeReservation* operation from *HotelChain* in order to illustrate the technique.

SAQ 10

Which parties are involved in satisfying the postcondition of a system operation? How can you identify them?

ANSWER.....

- The parties are:
- ▶ the actor that initiates the corresponding use case;
 - ▶ the system object, which orchestrates the collaboration;
 - ▶ a set of objects within the system which are required to satisfy the postcondition; these can be identified by looking at the operation specification.

Consider once again the operation *makeReservation*, now using value identities. Note that we have modified the postcondition to take into account the need to create a new *ReserverPayer* object in the case where it was not known to the hotel.

```
context HotelChain::makeReservation(hotelName : Name, startDate : Date, endDate :
Date, roomType : RoomType, name : Name, address : Address, creditCardDetails :
CreditCard) : String
pre:
-- startDate is before endDate
-- there is a Hotel with Name hotelName, and it has RoomTypes of roomType
```

post:

- if *available(startDate, endDate, roomType)* executed in the *Hotel* with *hotelName*
- returned true then
- a new *Reservation* object, *reservation* will have been created (object creation)
- if there is not a *ReserverPayer* with given details in that *Hotel* then
- a new *ReserverPayer* object will be created with attributes set to *name*, *address*,
- *creditCardDetails* and a newly generated *id*
- the new *ReserverPayer* object will be linked to *hotelChain*
- that *Hotel* will be linked to *reservation*
- the *ReserverPayer* will be linked to *reservation*
- *reservation* will be linked to *roomType*
- *reservation* will be linked to a previously free *Room*
- a string indicating the reservation *number* will have been returned
- otherwise, a string indicating failure will have been returned

SAQ 11

By looking at *makeReservation*'s postcondition, identify the objects which are involved in the collaboration, and their links.

ANSWER.....

Besides the system object, there is a *Hotel* object, a *ReserverPayer* object (either pre-existing or newly created), a *RoomType* object, a *Room* object and the newly created *Reservation* object.

Fulfilling the postcondition of the operation *makeReservation* corresponds to the creation of objects and the creation of new links between objects, as you have seen in the snapshot in Figure 8. To show this, we choose to use communication diagrams rather than sequence diagrams, as the former clearly show the creation of new objects and new links with the *{new}* constraint. Sequence diagrams are better when we need to be clear about the temporal ordering of messages.

The responsibility of the fulfilment of *makeReservation* will ultimately lie with *Hotel*, as each hotel has the expertise required to achieve this operation (GRASP Expert): a hotel knows about the reservations and people who made them, and about rooms and room types.

We have already identified one operation of the class *Hotel* that is relevant to fulfil the postcondition of *makeReservation*: the operation *available*. Its operation signature is as follows.

context *Hotel::available(startDate : Date, endDate : Date, roomType : RoomType) : Boolean*

We now need to discuss which class should take responsibility for the operation that creates a new *ReserverPayer*, and the operation that creates a new reservation. By applying GRASP Creator, the creation of a new guest should be assigned to *HotelChain*, as *HotelChain* records and holds information for all guests that ever come into contact with the hotel chain. As for a new reservation, the responsibility may be assigned to either *Hotel* or *Guest*: either of these classes may record information about a reservation. We decide to assign the responsibility to *Hotel*.

context HotelChain ::createReserverPayer(id : Id, name : Name, address : Address, creditCardDetails : CreditCard) : ReserverPayer

context Hotel::createReservation(startDate : Date, endDate : Date, roomType : RoomType, reserverPayer : ReserverPayer) : Reservation

Ultimately, an object is created by a class operation, usually known as the **constructor** operation; we use the convention of naming these class operations that create new instances as *create*. The constructor operation returns the newly created instance, that is, an object identity for the newly created object. The operations *createReserverPayer* and *createReservation* will invoke the *create* class operations of *ReserverPayer* and *Reservation* respectively.

context ReserverPayer::create(id : Id, name : Name, address : Address, creditCardDetails : CreditCard) : ReserverPayer

The class operation that creates a new *Reservation* will need to know not only the attributes of the new object, but also the object references of the objects that a new reservation should be linked to: the *ReserverPayer* and the *RoomType* for that reservation.

context Reservation::create(startDate : Date, endDate : Date, roomType : RoomType, reserverPayer : ReserverPayer) : Reservation

pre:

-- none

post:

-- a new *Reservation* is created and returned

-- its attributes are initialised to the given values

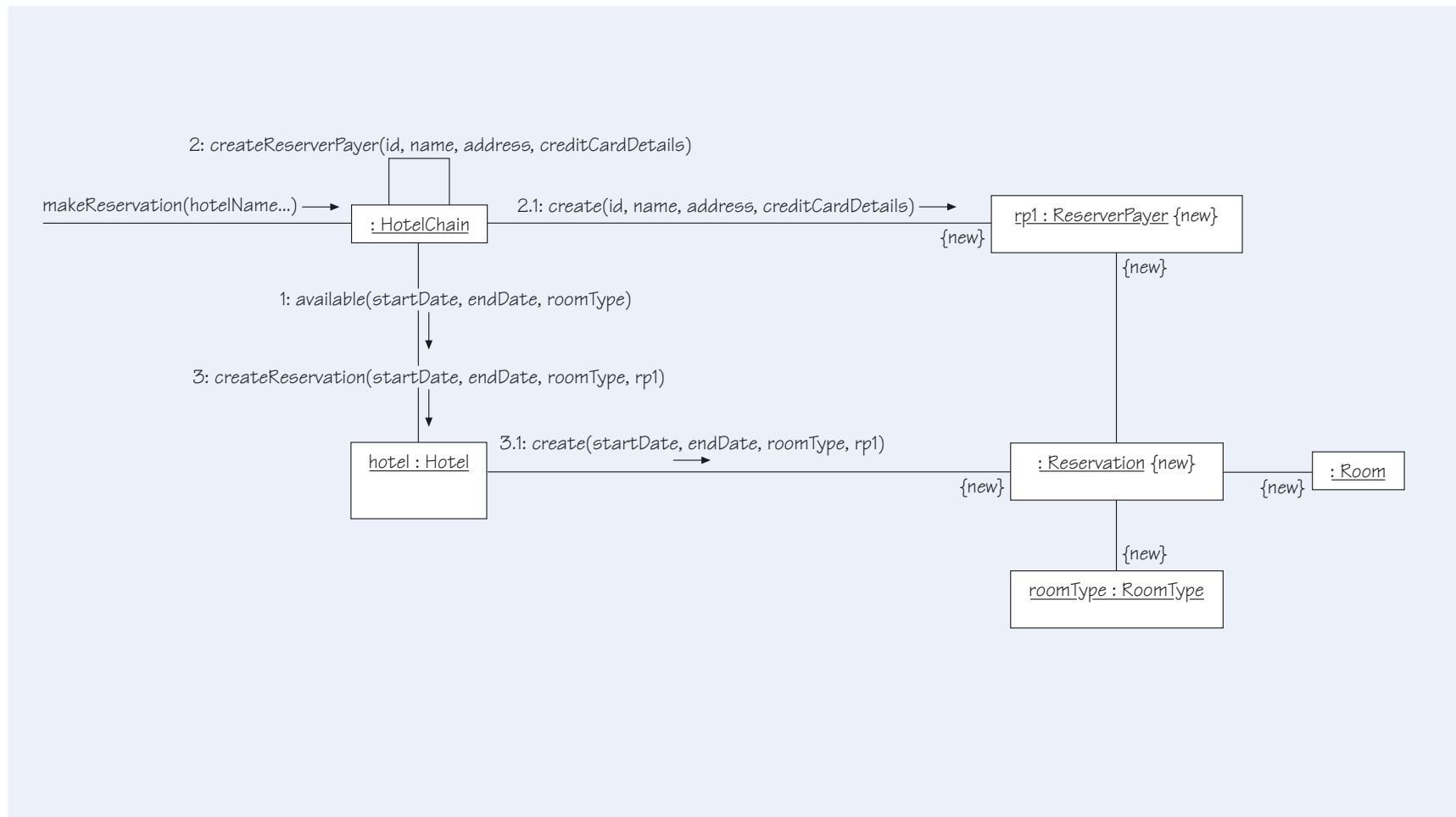
-- it is linked to the objects passed as parameters: *reserverPayer* and *roomType*

-- and a newly allocated room

The communication diagram of Figure 14 shows the interactions that fulfil the postcondition of *makeReservation* when *reserverPayer* is not known to the hotel, and when there is availability of the requested *roomType*. Note the annotation of the new objects and the new links with the constraint {*new*}. This is a detail not fully covered in our earlier treatment of communication diagrams. Note also the order in the messages, given by their numbering.

We have shown the need for a new link to the allocated room in the diagram, but controversially have left the details of how to achieve this as an implementation detail. Part of our rationale here is that the diagram provides sufficient detail to show our overall architectural approach.

Note that by annotating each new link with the constraint {*new*} next to one of the ends of the link, we are implying a direction of navigation for the corresponding associations – from *Hotel* to *Reservation*, from *HotelChain* to *ReserverPayer*, from *Reservation* to *ReserverPayer*, and from *Reservation* to *RoomType*. As you will see in Exercise 8, some of these directions are quite arbitrary at the moment. The direction of the association between *ReserverPayer* and *Reservation* could be the opposite, and you may decide, at some point, that such an association should be navigated in both directions – a *ReserverPayer* being able to access its *Reservations*, and a *Reservation* being able to access its *ReserverPayer*. There are naturally consequences of choosing the direction of the navigation of an association, but these should be defined in the context of all design operations. These decisions have to do with the resulting amount of coupling and cohesion, as discussed in *Unit 6*.

Figure 14 Communication diagram for the fulfilment of the postcondition of *makeReservation*

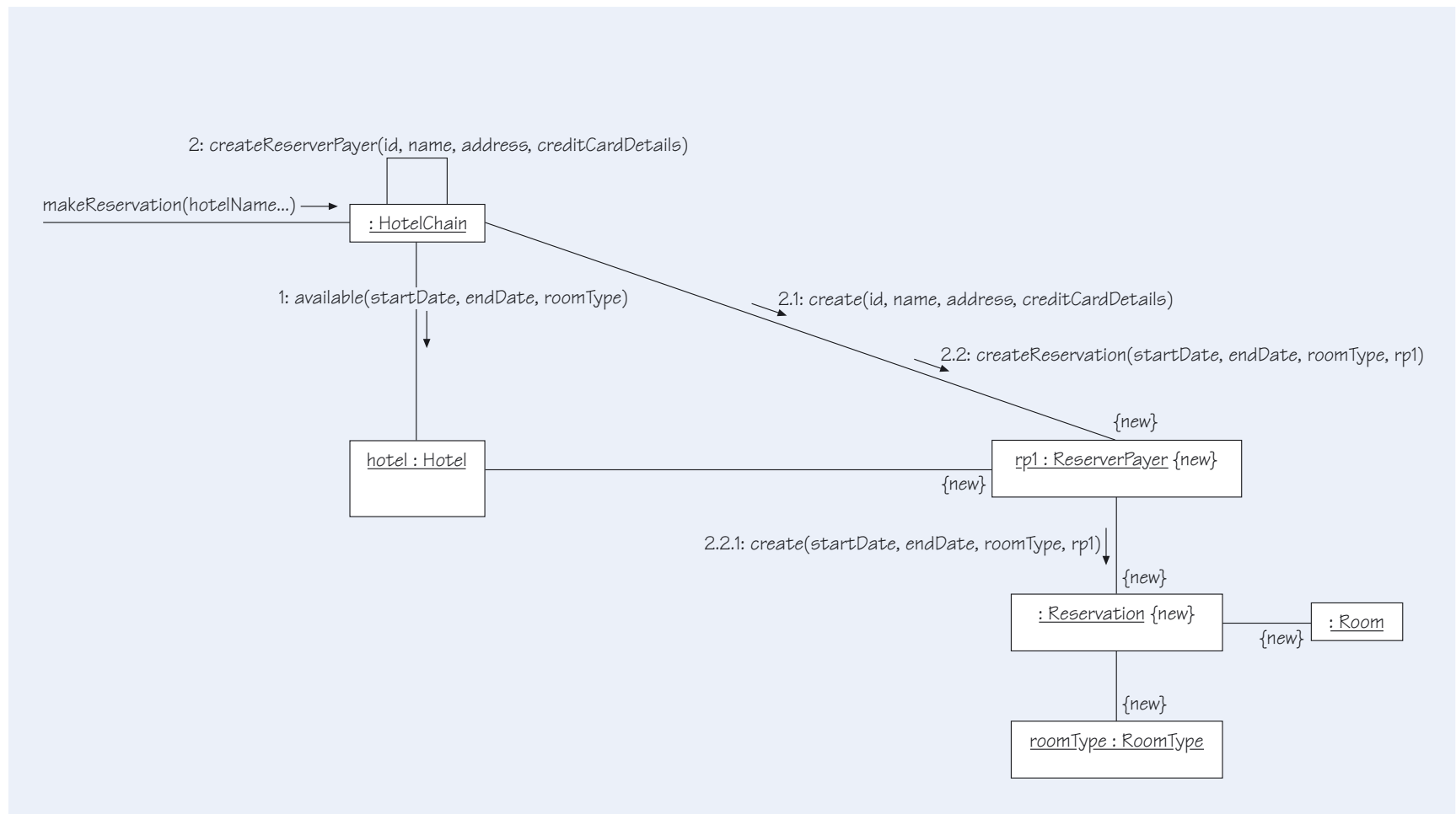


Figure 15 Revised communication diagram for the fulfilment of the postcondition of *makeReservation*

Exercise 8



Redraw the communication diagram of Figure 14 assuming that the responsibility for creating a reservation is assigned to *ReserverPayer*.

Solution
Our solution is shown in Figure 15.

Exercise 9



Redraw the communication diagram of Figure 14 assuming that the *ReserverPayer* is known to the *Hotel*.

Solution
Our solution is shown in Figure 16.

Now consider the *checkInGuest* operation again:

context HotelChain::checkInGuest(hotelName : Name, reservationNo : Integer, roomNo : Integer, name : Name, address : Address)
pre:
-- the *Reservation* with *reservationNo* is linked to the *Hotel* with *hotelName*
-- the *Room* with *roomNo* is linked to the *Reservation*, and the *Room* is of the *RoomType* for the *Reservation*
post:
-- if *name* is not that of the *ReserverPayer* of the *Reservation* with *reservationNo*, then
-- a new *Guest* will be created, and its attributes set; this new *Guest* or,
-- otherwise, the *ReserverPayer* will be linked to the *Room* with *roomNo*

SAQ 12

By looking at *checkInGuest*'s postcondition, identify the objects which are involved in the collaboration.

ANSWER.....
Besides the system object, there is a *Hotel* object, either a pre-existing *ReserverPayer* object or a newly created *Guest*, a *Room* object, and a *Reservation* object.

Exercise 10



- (a) Which class should have the responsibility for creating a new *Guest* object, and why?
- (b) Which new operations do you suggest should be added to fulfil the postcondition of *checkInGuest*?

Solution
(a) The responsibility for creating a *Guest* should be assigned to *Room*: it is the only class that is associated with *Guest*, and should therefore have this responsibility (GRASP Creator).
(b) The following operations need to be added:
Room::createGuest
Guest::create

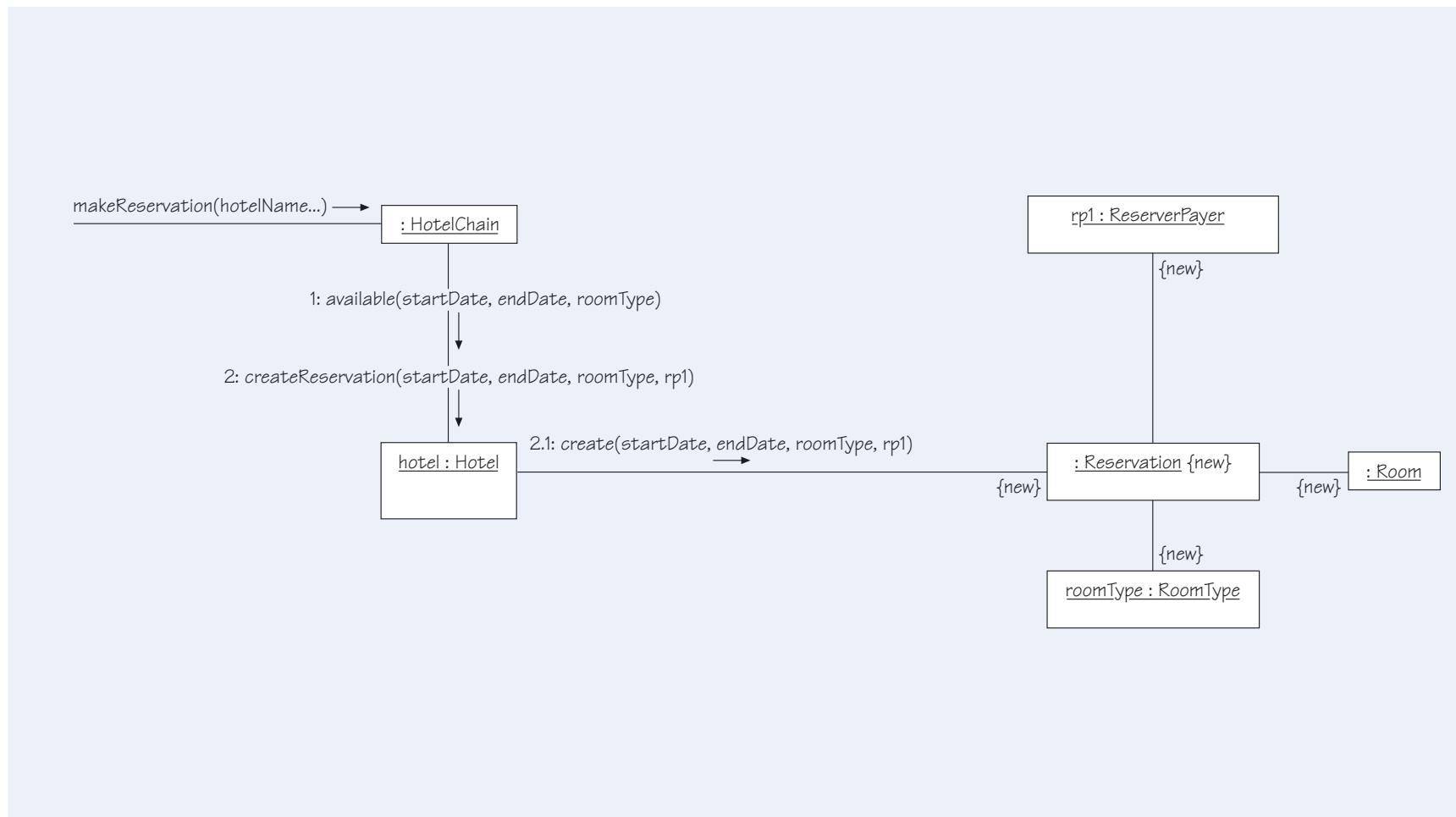


Figure 16 Revised communication diagram for *makeReservation*, assuming that the *ReserverPayer* is known to the *Hotel*

4.3 Revising the class diagram

We can now redraw the class diagram to take into account the allocation of operations to classes that we have carried out so far, along with some tentative decisions in relation to the navigation of operations.

We need to add the following operations to Figure 13. See Figure 17.

```
HotelChain::makeReservation  
HotelChain::cancelReservation  
HotelChain::checkInGuest  
HotelChain::checkOutGuest  
HoteChain::createReserverPayer  
HotelChain::canMakeReservation  
HotelChain::canCancelReservation  
HotelChain::canCheckInGuest  
HotelChain::canCheckOutGuest  
Hotel::createReservation  
Hotel::cancelReservation  
Hotel::checkInGuest  
Hotel::checkOutGuest  
Hotel::available  
Hotel::createReservation  
Room::createGuest  
ReserverPayer::create  
Reservation::create  
Guest::create
```

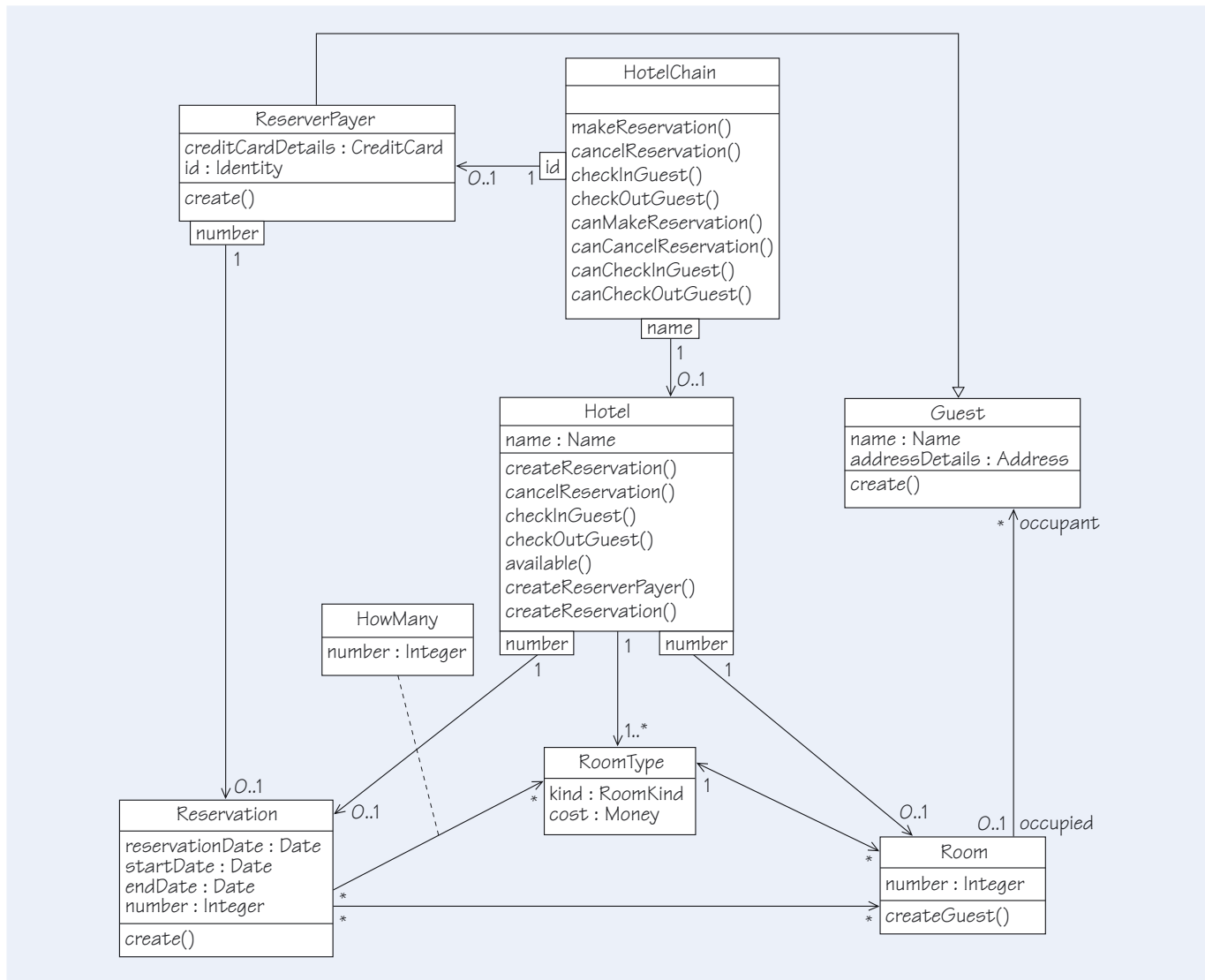


Figure 17 Class diagram with operations shown

4.4 Summary of section

In this section you have seen how system operations were allocated to classes, and how their pre- and postconditions are fulfilled. You also saw how navigability was added to the class diagram.

5

Summary

This unit has dealt with domain analysis and further analysis, looking at constraints on the structural model and the identification of the system operations. It has also covered design in terms of the allocation of responsibilities to classes.

LEARNING OUTCOMES

On completion of this unit you should be able to:

- ▶ carry out domain modelling, by identifying concepts, associations, multiplicities and attributes;
- ▶ carry out analysis, by specifying constraints and identifying system operations;
- ▶ carry out design, by identifying further system operations, establishing pre- and postconditions, and assigning system operations to classes.

Reference

Jacobson, I., Booch, G. and Rumbaugh, J. (1999) *The Unified Software Development Process*, Reading, MA, Addison-Wesley.

Index

A

accounting systems 11
analysis 5, 11
analysis class diagrams 17
analysis classes 5
analysis structural models 5
association classes 17, 19
association loop invariants 17
associations 11–12
attributes 11

B

bills 11
business processes 21

C

candidate classes 9
class diagrams 5, 8, 11, 13
class operations 38
classes 8
cohesion 38
communication diagrams 37
concept diagrams 5, 8, 17
constraint {new} 38
constraints 6, 17, 31
constructors 38
contexts of operations 21
coupling 38
Creator pattern (GRASP) 6, 37

D

derived attributes 17
design 5, 11
development processes 6
domain modelling 13, 17

E

events 8
Expert pattern (GRASP) 6
external interfaces 21

F

fulfilling postconditions 6

G

glossaries 5
GRASP 6

H

hotel 8

I

interaction diagrams 6
invoices 11

L

links 38

M

multiplicities 17

N

names of operations 21
navigation 38
navigation of associations 32

O

object identifiers 32
organisational units 8

P

parameters of operations 21
placeholders 14
postconditions 6, 21, 33
preconditions 6, 21, 33
properties 11

Q

qualified associations 32

R

roles 8

S

sequence diagrams 37
signatures 21
structural domain models 5
structural models 5, 31
system classes 21
system operations 6, 21, 31

T

tangible objects 8

U

Unified Process (UP) 5
use cases 6, 31

V

value identities 32
verifying preconditions 6